

Arrays & Algorithmic Problem Solving

Partitioning, Custom Sorting, Binary Search, Two Pointers, Sliding Windows & Discrete Calculus

Session Agenda

Idea 1: The Partition Engine

From Bubble Sort to QuickSort & $\mathcal{O}(N)$ In-Place Segregation

Idea 2: Custom Sorting & Comparators

Extending Order: Multi-Field Structs, Tiebreakers & Invariants

Idea 3: Searching, Bounds & Optimization

Binary Search, Lower/Upper Bounds & Monotonic Predicates on Answer

Idea 4: Two-Pointer Techniques

Converging Invariants, Greedy Elimination & State Space Pruning

Idea 5: Sliding Window Technique

Fixed vs. Variable Windows, Invariant Expansion/Contraction & Deques

Idea 6: Prefix Sums & Difference Arrays

Discrete Calculus on Arrays, $\mathcal{O}(1)$ Range Queries & Boundary Impulses

Idea 1

The Partition Engine

From Bubble Sort to QuickSort & $\mathcal{O}(N)$ In-Place Segregation

Chapter 01: Arrays | Interview Problem Solving Masterclass

The Problem: Sorting an Array

The Challenge

Given an unsorted array of N integers:

[5, 2, 8, 3, 1]

Rearrange them in non-decreasing order:

[1, 2, 3, 5, 8]

Where Do We Begin?

What is the most natural, brute-force way to sort?

Can we inspect elements locally—comparing immediate adjacent neighbors—and gradually bring order to the array?

The Intuition: Bubbles in a Lake

The Physical Intuition

Imagine air bubbles underwater in a calm lake. Because larger bubbles experience greater buoyancy, the **largest bubble naturally rises to the surface first**. In an array, if you repeatedly compare adjacent elements and swap whenever the left is larger, the **maximum element bubbles to the tail!**

The Pass-by-Pass Guarantee

- **Pass 1:** Pushes the **1st largest** to index $N - 1$.
- **Pass 2:** Pushes the **2nd largest** to index $N - 2$.
- **Pass k :** Pushes the **k -th largest** to index $N - k$.
- After $N - 1$ passes, the entire array is sorted!

Invariant

Each pass scans the array and locks the largest remaining element at the end.

Bubble Sort: The Java Implementation

```
1 void bubbleSort(int[] arr) {  
2     int n = arr.length;  
3     for (int i = 0; i < n - 1; i++) {  
4         for (int j = 0; j < n - 1 - i; j++) {  
5             if (arr[j] > arr[j + 1]) {  
6                 int temp = arr[j];  
7                 arr[j] = arr[j + 1];  
8                 arr[j + 1] = temp;  
9             }  
10        }  
11    }  
12 }
```

Loop Invariants

Outer Loop (i): Tracks completed passes. After pass i , the suffix subarray $arr[n - i..n - 1]$ is sorted.

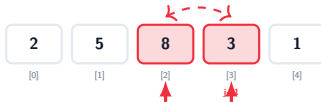
Inner Loop (j): Sweeps through the unsorted prefix (0 to $n - 2 - i$), bubbling the current largest value rightward.

Bubble Sort: Pass 1 Execution

Step 1: Compare $\text{arr}[0]$ (5) > $\text{arr}[1]$ (2) $\rightarrow$ Swap!



Step 2: Compare $\text{arr}[2]$ (8) > $\text{arr}[3]$ (3) $\rightarrow$ Swap!



End of Pass 1: Element 8 is locked permanently at index [4]!



Invariant

Pass 1 finishes: The maximum element 8 is in its exact final position!

Time Complexity Analysis: Bubble Sort

Counting Comparisons

- Pass 1: $N - 1$ comparisons
- Pass 2: $N - 2$ comparisons
- Pass k : $N - k$ comparisons

$$\text{Total} = \sum_{k=1}^{N-1} (N - k) = \frac{N(N - 1)}{2}$$

$\mathcal{O}(N^2)$

Quadratic Time Complexity

On $N = 10^5$, $\sim 5 \times 10^9$ operations $\Rightarrow$ Time Limit Exceeded!

Space Complexity: $\mathcal{O}(1)$ auxiliary space (all swaps occur in-place).

Invariant

Each pass does linear work just to lock ONE element. Can we do significantly better?

The Partition Concept: Placing the Pivot

The Core Insight

Instead of slow adjacent bubbles, pick an element: the **Pivot** (e.g. `arr[high]`).

In a **single linear pass** ($\mathcal{O}(N)$), rearrange the array so:

- All elements $\leq$ Pivot are to its **left**.
- All elements $>$ Pivot are to its **right**.

Subarray Decomposition

The pivot lands in its **exact final sorted index** (`pIndex`) in only $\mathcal{O}(N)$ time!

We now have two independent smaller subproblems:

1. Left subarray: `arr[low..pIndex - 1]`
2. Right subarray: `arr[pIndex + 1..high]`

Invariant

One partition locks the pivot and cuts the remaining sorting problem into two!

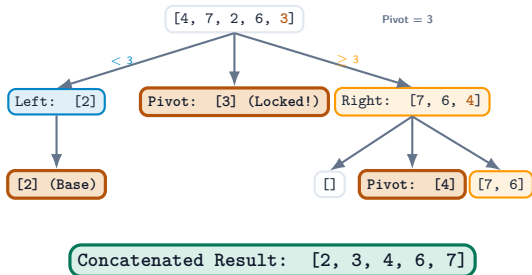
QuickSort: Divide & Conquer

```
1 void quickSort(int[] arr, int low, int high) {  
2     if (low < high) {  
3         int pIndex = partition(arr, low, high);  
4         quickSort(arr, low, pIndex - 1);  
5         quickSort(arr, pIndex + 1, high);  
6     }  
7 }
```

Divide & Conquer Recipe

1. **Divide:** Call `partition()` to place pivot at `pIndex`.
2. **Conquer:** Recurse on `[low..pIndex - 1]` and `[pIndex + 1..high]`.
3. **Combine:** Zero work needed! When both subarrays are sorted, the whole array is sorted.

Visualizing the Recursive Split Tree



$$O(\log_2 N)$$

Recursion Tree Depth

Problem size roughly halves at each level on average

$$O(N \log N)$$

Average Time Complexity

N total work per level $\times \log N$ tree depth

The Real Question: How Does Partition Work?

The Subroutine Mystery

QuickSort's entire efficiency rests on:

```
pIndex = partition(arr, low, high)
```

How can we rearrange elements around the pivot in a **single linear pass** ($\mathcal{O}(N)$) without allocating extra memory?

The In-Place Requirement

Creating separate auxiliary arrays takes $\mathcal{O}(N)$ extra memory per call.

We need an **in-place two-pointer technique** that partitions the array directly using element swaps!

Lomuto Partition: Pointer Invariants

Index i : Smaller Subarray Boundary

Role: Demarcates the boundary of elements smaller than the pivot.

Every element in `arr[low .. i-1]` is strictly smaller than `pivot`.

Index j : Current Scanning Pointer

Role: Scans unvisited elements from `low` to `high - 1`.

Compares each element against `pivot` to decide if it belongs in the left partition.

The Invariant Swap Rule: When `arr[j] < pivot`, swap `arr[i]` with `arr[j]` and increment boundary `i`!

Lomuto Partition: Java Implementation

```
1 int partition(int[] arr, int low, int high) {
2     int pivot = arr[high];
3     int i = low;
4     for (int j = low; j < high; j++) {
5         if (arr[j] < pivot) {
6             swap(arr, i, j);
7             i++;
8         }
9     }
10    swap(arr, i, high);
11    return i;
12 }
```

Step-by-Step Logic

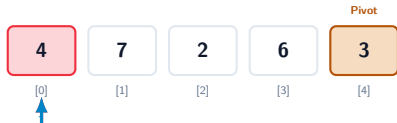
Lines 5–8: Pointer j checks if $\text{arr}[j] < \text{pivot}$. If true, swap $\text{arr}[i] \leftrightarrow \text{arr}[j]$ and increment i .

Line 10: Swap pivot ($\text{arr}[\text{high}]$) with $\text{arr}[i]$.

Line 11: Return index i as the permanent pivot location.

Lomuto Partition: Element Greater Than Pivot

```
1 int partition(int[] arr, int low, int high) {  
2     int pivot = arr[high]; // pivot = 3  
3     int i = low;          // i = 0  
4     for (int j = low; j < high; j++) {  
5         if (arr[j] < pivot) { // 4 < 3 (FALSE)  
6             swap(arr, i, j);  
7             i++;  
8         }  
9     }  
10    swap(arr, i, high);  
11    return i;  
12 }
```



State: $i=0, j=0$ | $arr[j]=4, pivot=3$ | $4 < 3$
(FALSE)

Step Deduction – Element Greater Than Pivot

$arr[0] = 4 \geq 3$; belongs to the $\geq$ partition. No swap performed. Boundary i remains at 0; index j advances to 1.

Lomuto Partition: Maintaining Partition Boundary

```
1 int partition(int[] arr, int low, int high) {  
2     int pivot = arr[high]; // pivot = 3  
3     int i = low;           // i = 0  
4     for (int j = low; j < high; j++) {  
5         if (arr[j] < pivot) { // 7 < 3 (FALSE)  
6             swap(arr, i, j);  
7             i++;  
8         }  
9     }  
10    swap(arr, i, high);  
11    return i;  
12 }
```



State: $i=0, j=1$ | $\text{arr}[j]=7, \text{pivot}=3$ | $7 < 3$
(FALSE)

Step Deduction – Second Element Greater Than Pivot

$\text{arr}[1] = 7 \geq 3$. Both 4 and 7 are in $\geq$ partition.
No swap performed; boundary i stays at 0; index j advances to 2.

Lomuto Partition: Smaller Element Swap

```
1 int partition(int[] arr, int low, int high) {
2     int pivot = arr[high]; // pivot = 3
3     int i = low;          // i = 0
4     for (int j = low; j < high; j++) {
5         if (arr[j] < pivot) { // 2 < 3 (TRUE)
6             swap(arr, i, j); // swap arr[0] <-> arr[2]
7             i++;             // i -> 1
8         }
9     }
10    swap(arr, i, high);
11    return i;
12 }
```



State: swap(arr, 0, 2) | i -> 1

Step Deduction – Smaller Element Swapped to Front

Because $\text{arr}[2] = 2 < 3$, swap $\text{arr}[0]$ with $\text{arr}[2]$. The smaller element is placed in the left partition, and boundary i advances to 1!

Lomuto Partition: Final Pivot Placement

```
1 int partition(int[] arr, int low, int high) {  
2     int pivot = arr[high]; // pivot = 3  
3     int i = low;           // i = 1  
4     for (int j = low; j < high; j++) {  
5         // ... loop completed ...  
6     }  
7     swap(arr, i, high);    // swap arr[1] <-> arr[4]  
8     return i;              // returns pIndex = 1  
9 }
```



Final Lock: `swap(arr, 1, 4) | pIndex = 1`

Step Deduction – Pivot in Final Sorted Position

Pivot 3 now sits at index 1. All elements to its left ([2]) are smaller; all elements to its right ([4, 6, 7]) are larger!

Time and Space Complexity: Lomuto Partition

$$\mathcal{O}(N)$$

Linear Time Complexity

Pointer j performs exactly $N - 1$ comparisons in a single pass.

$$\mathcal{O}(1)$$

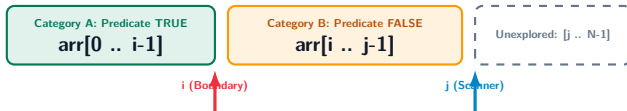
In-Place Space Complexity

All rearrangements happen via index swaps; zero auxiliary memory allocated.

Invariant

In a single linear pass with $\mathcal{O}(1)$ auxiliary space, the array is partitioned!

Generalization: In-Place Partitioning by Predicate



The Partition Invariant

Whenever you want all elements satisfying property $P(x)$ on the left:

Scanner j checks if $P(arr[j])$ is TRUE and swaps it into boundary index i .

Applications of Predicate Partitioning

1. **Signs:** Negative elements on left, positives on right.
2. **Zeros:** Non-zero elements on left, zeros on right.
3. **Parity:** Even integers on left, odd integers on right.

Invariant

Partition is a general-purpose $\mathcal{O}(N)$ in-place segregation technique!

Variation 1: Segregating Negatives and Positives

```
1 void segregateNegatives(int[] arr) {  
2     int i = 0;  
3     for (int j = 0; j < arr.length; j++) {  
4         if (arr[j] < 0) { // Partition around 0  
5             swap(arr, i, j);  
6             i++;  
7         }  
8     }  
9 }
```

Partitioning Around Zero

The pivot does not need to be an element from the array!

Treat 0 as a fixed threshold value. Every negative element is swapped into the left subarray $[0..i-1]$.

Variation 2: Move All Zeros to the End

```
1 void moveZeroes(int[] arr) {  
2     int i = 0;  
3     for (int j = 0; j < arr.length; j++) {  
4         if (arr[j] != 0) { // Keep non-zeros on left  
5             swap(arr, i, j);  
6             i++;  
7         }  
8     }  
9 }
```

Non-Zero Predicate

Collect all **non-zero elements** at the front!

When all non-zeros are gathered into $[0..i - 1]$, all zero elements are naturally placed at the tail $[i..N - 1]$.

Variation 3: Segregating Even and Odd Numbers

```
1 void segregateEvenOdd(int[] arr) {  
2     int i = 0;  
3     for (int j = 0; j < arr.length; j++) {  
4         if ((arr[j] & 1) == 0) { // Bitwise parity  
5             swap(arr, i, j);  
6             i++;  
7         }  
8     }  
9 }
```

Bitwise Parity Check

In Java and C++, `arr[j] % 2 != 0` can return -1 for negative odd numbers.

Use `(arr[j] & 1) == 0` for a robust, branch-free parity check!

Summary: The Partition Template

Same Loop · Same Pointers · Same Swaps · Only 1 Line Changes!

Problem Statement	Predicate Condition	Left Partition	Right Partition
QuickSort Partition	<code>arr[j] < pivot</code>	Elements < pivot	Elements $\geq$ pivot
Positive vs Negative	<code>arr[j] < 0</code>	Negative numbers	Positive numbers
Move Zeros to End	<code>arr[j] != 0</code>	Non-zero elements	Trailing zeros
Even vs Odd Parity	<code>(arr[j] & 1) == 0</code>	Even integers	Odd integers

Invariant

Mastering the partition algorithm provides an in-place $O(N)$ solution to an entire class of array problems!

Idea 2

Custom Sorting & Comparators

Extending Order: Multi-Field Structs, Tiebreakers & Invariants

Chapter 01: Arrays | Interview Problem Solving Masterclass

The Problem: What Does “Sorting” Really Mean?

Beyond Natural Order

For integers, sorting is intuitive: values have an inherent numerical order ($1 < 2 < 3$).

But how should we sort:

- **Strings?** By alphabetical order, or by length?
- **2D Points** (x, y)? By X , by Y , or by distance from origin?
- **Custom Objects?** By multiple prioritized attributes?

The Core Question

Sorting is not fixed to numerical values.

An algorithm only needs an answer to one fundamental question:

“Between element A and element B , which one should appear first?”

Invariant

A custom sort allows us to define total ordering over any domain by specifying a comparison rule.

Built-in Sorting Baseline: Java Arrays.sort

```
1 // 1. Natural Numerical Order
2 Integer[] nums = {5, 2, 8, 1};
3 Arrays.sort(nums);
4 // Output: [1, 2, 5, 8]
5
6 // 2. Default Lexicographical Order
7 String[] words = {"banana", "apple", "fig", "kiwi"};
8 Arrays.sort(words);
9 // Output: ["apple", "banana", "fig", "kiwi"]
```

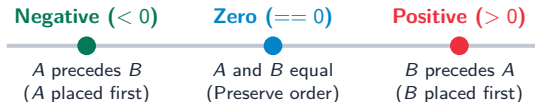
Default Inbuilt Behavior

Primitives / Numbers: Sorted in natural non-decreasing order.

Strings: Sorted lexicographically (dictionary order) comparing characters by ASCII values left-to-right.

Limitation: Cannot sort strings by length or points by coordinates without a custom rule!

The Comparator Contract: The 3-State Rule



Java Comparator<T> Interface

`int compare(T a, T b)`

- **Negative (< 0):** a is strictly smaller $\implies a$ precedes b .
- **Zero ($= 0$):** a and b are equivalent.
- **Positive (> 0):** a is strictly greater $\implies b$ precedes a .

Transitivity & Consistency

The comparison rule must define a valid **Total Order**:

- **Antisymmetry:**
 $\text{sgn}(\text{cmp}(a, b)) = -\text{sgn}(\text{cmp}(b, a))$
- **Transitivity:** If $\text{cmp}(a, b) < 0$ and $\text{cmp}(b, c) < 0$, then $\text{cmp}(a, c) < 0$.

Variation 1: Sorting Strings by Length

The Goal

Given an array of strings:

```
["banana", "apple", "fig", "kiwi"]
```

Rearrange them so **shorter strings appear first**:

```
["fig", "kiwi", "apple", "banana"]
```

Lengths: $3 \rightarrow 4 \rightarrow 5 \rightarrow 6$.

Extracting the Comparison Key

Instead of comparing character ASCII values, we extract the attribute of interest:

$$\text{Key}(s) = s.\text{length}()$$

To sort in ascending order:

$$\text{cmp}(a, b) = a.\text{length}() - b.\text{length}()$$

String Length Comparator: Java Implementation

```
1 String[] arr = {"banana", "apple", "fig", "kiwi"};
2
3 // Custom Comparator: Sort by length
4 Arrays.sort(arr, (a, b) ->
5     Integer.compare(a.length(), b.length()));
6
7 // Output: ["fig", "kiwi", "apple", "banana"]
```

Why Integer.compare?

Never write `a.length() - b.length()` for arbitrary integers, as numerical subtraction can cause **integer overflow** with extreme negative values.

Always use standard library utilities like `Integer.compare(x, y)` or `Long.compare(x, y)`!

The Conflict: Resolving Ties

When Primary Keys Collide

What if two strings have the exact same length?

"kiwi" (len 4) vs "pear" (len 4)

Which one should appear first in the output?

- `cmp(a, b) == 0` leaves the order ambiguous.
- Real-world requirements demand deterministic ordering!

Two Tiebreaking Strategies

1. **Multi-Level Secondary Attribute:** If lengths are equal, sort alphabetically (lexicographical tiebreaker).
2. **Input Order Preservation (Stability):** If lengths are equal, maintain original position from input.

Invariant

All tiebreaking logic is expressed entirely inside the comparator!

Tiebreaker 1: Multi-Level Ordering (Length → Lexicographical)

```
1 String[] words = {"kiwi", "pear", "fig", "apple", "plum"};
2
3 Arrays.sort(words, (a, b) -> {
4     // Primary: Compare by length
5     if (a.length() != b.length()) {
6         return Integer.compare(a.length(), b.length());
7     }
8     // Secondary: Lexicographical tiebreaker
9     return a.compareTo(b);
10 });
11 // Output: ["fig", "kiwi", "pear", "plum", "apple"]
```

Cascading Logic

Primary Check: If lengths differ, immediately return the length comparison.

Fallback Check: If lengths are identical, invoke `a.compareTo(b)` to break the tie alphabetically.

Tiebreaker 2: Sorting Stability & The Index-Pair Technique

```
1 class Element {
2     String value;
3     int originalIndex;
4     Element(String v, int idx) {
5         this.value = v;
6         this.originalIndex = idx;
7     }
8 }
9 // Sort by length, tiebreak by originalIndex
10 Arrays.sort(elements, (a, b) -> {
11     int lenCmp = Integer.compare(
12         a.value.length(), b.value.length());
13     if (lenCmp != 0) return lenCmp;
14     return Integer.compare(
15         a.originalIndex, b.originalIndex);
16 });
```

Guaranteeing Stability

Definition of Stability: Elements with equal keys retain their relative initial order.

The Index-Pair Technique: Wrapping each item with its initial index guarantees deterministic stability across **any** sorting routine!

Variation 2: Sorting 2D Points by Distance from Origin

The Challenge

Given a collection of 2D points:

$$P_1(1, 1), P_2(-1, 2), P_3(2, -1)$$

Sort them in ascending order of their Euclidean distance from origin (0, 0):

$$d = \sqrt{x^2 + y^2}$$

Avoiding the sqrt Trap

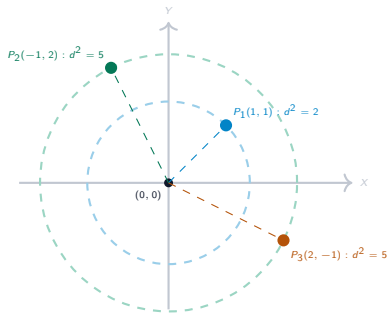
Because $f(z) = \sqrt{z}$ is strictly monotonically increasing for $z \geq 0$:

$$\sqrt{x_1^2 + y_1^2} < \sqrt{x_2^2 + y_2^2} \iff x_1^2 + y_1^2 < x_2^2 + y_2^2$$

Comparing squared distances eliminates precision loss and floating-point overhead!

2D Points Comparator: Java Implementation

```
1 class Point {  
2     int x, y;  
3     Point(int x, int y) {  
4         this.x = x; this.y = y;  
5     }  
6 }  
7  
8 Point[] pts = {  
9     new Point(1, 1),  
10    new Point(-1, 2),  
11    new Point(2, -1)  
12 };  
13  
14 Arrays.sort(pts, (p1, p2) -> {  
15     long d1 = 1L*p1.x*p1.x + 1L*p1.y*p1.y;  
16     long d2 = 1L*p2.x*p2.x + 1L*p2.y*p2.y;  
17     return Long.compare(d1, d2);  
18 });
```



Variation 3: Forming the Largest Number

The Challenge

Given a list of non-negative integers:

[3, 30, 34, 5, 9]

Arrange them such that when concatenated, they form the **largest possible number**:

"9534330"

Initial Intuition

Treat numbers as strings and sort in descending lexicographical order ($9 \rightarrow 5 \rightarrow 34 \rightarrow \dots$).

The Trap: How do we order strings with shared prefixes like 9 vs 98, or 30 vs 3?

Why Simple String Sorting Fails: The Prefix Conflict

Counterexample 1: [9, 98]

Standard string comparison: "98" > "9".

- Putting "98" first: "989"
- Putting "9" first: "998"

⇒ "9" must precede "98"!

Counterexample 2: [68, 6]

Standard string comparison: "68" > "6".

- Putting "68" first: "686"
- Putting "6" first: "668"

⇒ "68" must precede "6"!

Invariant

Neither string length nor simple prefix comparison works universally. We need a new invariant!

The Concatenation Invariant: $A + B$ vs $B + A$

Pair (9, 98): $A + B = \text{"998"}$ vs $B + A = \text{"989"}$ $\implies$ 9 precedes 98

Pair (68, 6): $A + B = \text{"686"}$ vs $B + A = \text{"668"}$ $\implies$ 68 precedes 6

The Master Decision Rule

For any two string representations A and B :

$$\text{order}(A, B) \implies (B + A).\text{compareTo}(A + B)$$

If $A + B > B + A$, placing A before B produces a strictly larger number!

Mathematical Transitivity

This pairwise concatenation rule forms a valid **total order**:

$$\text{If } A + B \geq B + A \text{ and } B + C \geq C + B$$

$$\implies A + C \geq C + A$$

Global sorting resolves the entire array optimally!

Largest Number: Java Implementation

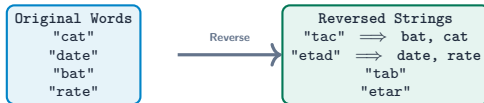
```
1 String largestNumber(int[] nums) {  
2     String[] strs = new String[nums.length];  
3     for (int i = 0; i < nums.length; i++) {  
4         strs[i] = String.valueOf(nums[i]);  
5     }  
6     // Sort descending by concatenation: (b+a) vs (a+b)  
7     Arrays.sort(strs, (a, b) -> (b + a).compareTo(a + b));  
8  
9     // Edge case: Leading zeroes (e.g. [0, 0])  
10    if (strs[0].equals("0")) return "0";  
11  
12    return String.join("", strs);  
13 }
```

Key Takeaways

Line 7: Custom comparator compares $(b + a)$ against $(a + b)$.

Line 10: If the largest element after sorting is "0", return "0" directly to prevent "00".

Variation 4: Suffix Sorting for Rhyming Words



The Rhyme Problem

Standard lexicographical sort compares strings **left-to-right** (matching prefixes).

Rhyming words share identical **suffixes** (endings).

The Reversal Technique

By comparing the **reversed** representations in the comparator, suffix matching naturally transforms into standard prefix sorting!

Rhyming Words Comparator: Java Implementation

```
1 String[] words = {"rate", "bat", "date", "cat", "mat"};
2
3 Arrays.sort(words, (a, b) -> {
4     String revA = new StringBuilder(a).reverse().toString();
5     String revB = new StringBuilder(b).reverse().toString();
6     return revA.compareTo(revB);
7 });
8 // Output: ["bat", "cat", "mat", "date", "rate"]
```

Mechanism

"cat" → "tac"

"bat" → "tab"

"date" → "etad"

"rate" → "etar"

All "-at" endings cluster together, followed by all "-ate" endings!

Time & Space Complexity of Custom Sorting

$$\mathcal{O}(N \log N \cdot K)$$

Time Complexity

Java's TimSort performs $\mathcal{O}(N \log N)$ comparisons. If comparator takes $\mathcal{O}(K)$ time (e.g. string length K), total time is $\mathcal{O}(K \cdot N \log N)$.

$$\mathcal{O}(N) \text{ or } \mathcal{O}(\log N)$$

Space Complexity

TimSort requires $\mathcal{O}(N)$ auxiliary space for object references; primitive arrays use $\mathcal{O}(\log N)$ recursion stack space.

Invariant

Always ensure the comparator performs $\mathcal{O}(1)$ or minimal $\mathcal{O}(K)$ work to avoid bottlenecking the $\mathcal{O}(N \log N)$ engine.

Summary: The Custom Comparator Matrix

Any Domain · Any Property · Defined by One Comparator Function

Problem Domain	Custom Comparator Condition	Order Principle
Default Primitives	<code>Integer.compare(a, b)</code>	Natural non-decreasing
Strings by Length	<code>Integer.compare(a.length(), b.length())</code>	Shorter strings first
Multi-Level Tiebreaker	<code>len != len ? lenCmp : a.compareTo(b)</code>	Length, then alphabetical
2D Point Distance	<code>Long.compare(x1*x1+y1*y1, x2*x2+y2*y2)</code>	Squared radial distance
Largest Number	<code>(b + a).compareTo(a + b)</code>	Concatenation precedence
Rhyming Suffixes	<code>revA.compareTo(revB)</code>	Suffix-grouped order

Invariant

By formulating the comparison invariant, built-in sorting solves complex multidimensional ordering problems in $\mathcal{O}(N \log N)$ time!

Idea 3

Searching, Bounds & Optimization

Binary Search, Lower/Upper Bounds & Monotonic Predicates on Answer

Chapter 01: Arrays | Interview Problem Solving Masterclass

The Problem: Searching in an Ordered Domain

The Search Problem

Given a collection of N sorted elements and a query key K , determine the index where K resides, or report its absence.

- **Linear Scan ($\mathcal{O}(N)$):** Inspects elements sequentially from index 0 to $N - 1$.
- **The Bottleneck:** For $N = 10^9$, a linear scan requires 10^9 operations (≈ 1.0 s).
- **The Fundamental Question:** Can we exploit the **monotonic sorted structure** to search exponentially faster?

The 50% Elimination Invariant

Because the array is sorted, comparing target K against the middle element $\text{arr}[\text{mid}]$ eliminates exactly **half** of the remaining candidates in a single comparison.



The Intuition: Search Space Halving

The Physical Dictionary Analogy

When looking up **"Prefix"** in a dictionary:

- You do not read page 1, page 2, page 3...
- You open directly to the middle.
- If you see **"Mathematics"**, you immediately discard the entire left half.
- You recurse on the remaining right half.

Logarithmic Collapse

Each comparison cuts the search window in half:

$$N \rightarrow \frac{N}{2} \rightarrow \frac{N}{4} \rightarrow \dots \rightarrow 1 \implies \lceil \log_2 N \rceil \text{ steps}$$

Step	Search Space Size	Comparisons
0	$N = 10^9$ (Billion)	1
10	$\approx 10^6$ (Million)	10
20	$\approx 10^3$ (Thousand)	20
30	≤ 1 (Element)	≤ 30

Binary Search: The Java Implementation

```
1 public static int binarySearch(int[] arr, int target) {
2     int low = 0, high = arr.length - 1;
3     while (low <= high) {
4         int mid = low + (high - low) / 2;
5         if (arr[mid] == target) {
6             return mid; // Target found
7         } else if (arr[mid] < target) {
8             low = mid + 1; // Search right half
9         } else {
10            high = mid - 1; // Search left half
11        }
12    }
13    return -1; // Target absent
14 }
```

Implementation Mechanics

Line 4: Safe midpoint $\text{low} + \frac{\text{high} - \text{low}}{2}$ prevents 32-bit signed integer overflow.

Lines 7 & 9: Strictly contract the search boundary by setting $\text{low} = \text{mid} + 1$ or $\text{high} = \text{mid} - 1$ to guarantee termination.

Binary Search: Step-by-Step Search Space Reduction

Query Trace

Target: 23

- Pass 1: $L = 0, H = 9 \implies \text{mid} = 4$
 $\text{arr}[4] = 16 < 23 \implies L = 5.$
- Pass 2: $L = 5, H = 9 \implies \text{mid} = 7$
 $\text{arr}[7] = 56 > 23 \implies H = 6.$
- Pass 3: $L = 5, H = 6 \implies \text{mid} = 5$
 $\text{arr}[5] = 23 == 23 \implies$ **Found at index 5!**

Pass 1: Mid at Index 4 (Value 16)



Pass 2: Left Discarded; Mid at Index 7 (Value 56)



Pass 3: Right Discarded; Mid at Index 5 (Value 23)



Time Complexity Analysis: Binary Search

Recurrence Derivation

At each step, we perform $\mathcal{O}(1)$ work and recurse on half the subarray:

$$T(N) = T\left(\frac{N}{2}\right) + \mathcal{O}(1)$$

By Master Theorem:

$$T(N) = \mathcal{O}(\log_2 N)$$

Space Complexity: $\mathcal{O}(1)$ auxiliary memory for iterative implementation.

Growth Comparison

Domain (N)	Linear $\mathcal{O}(N)$	Binary $\mathcal{O}(\log N)$
10^3 (Thousand)	10^3 ops	≈ 10 ops
10^6 (Million)	10^6 ops	≈ 20 ops
10^9 (Billion)	10^9 ops	≈ 30 ops
10^{18} (Quintillion)	TLE	≈ 60 ops

Logarithmic time transforms massive domains into sub-microsecond computations.

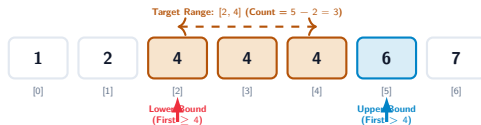
Variation 1: Lower Bound and Upper Bound

The Duplicate Key Challenge

In an array with duplicates (e.g. $[1, 2, 4, 4, 4, 6, 7]$), standard binary search returns an arbitrary match.

- **Lower Bound:** Smallest index i where $\text{arr}[i] \geq \text{target}$.
- **Upper Bound:** Smallest index i where $\text{arr}[i] > \text{target}$.
- **Frequency Count:**

$$\text{Count} = \text{upper_bound} - \text{lower_bound}$$



The Mindset Shift

Do not stop upon finding an equality match. **Record the candidate index** and continue shrinking towards the boundary!

Lower Bound: Java Implementation

```
1 public static int lowerBound(int[] arr, int target) {
2     int low = 0, high = arr.length - 1;
3     int ans = arr.length; // Default insertion point
4     while (low <= high) {
5         int mid = low + (high - low) / 2;
6         if (arr[mid] >= target) {
7             ans = mid; // Candidate found
8             high = mid - 1; // Search leftward
9         } else {
10             low = mid + 1; // Search rightward
11         }
12     }
13     return ans;
14 }
```

Boundary Logic

Lines 6–7: When $\text{arr}[\text{mid}] \geq \text{target}$, mid is a valid candidate. We record $\text{ans} = \text{mid}$ and compress $\text{high} = \text{mid} - 1$ to check for earlier occurrences.

Default: If no element is $\geq \text{target}$, returns N (the natural insertion index).

Upper Bound: Java Implementation

```
1 public static int upperBound(int[] arr, int target) {
2     int low = 0, high = arr.length - 1;
3     int ans = arr.length; // Default insertion point
4     while (low <= high) {
5         int mid = low + (high - low) / 2;
6         if (arr[mid] > target) {
7             ans = mid; // Candidate found
8             high = mid - 1; // Search leftward
9         } else {
10             low = mid + 1; // Search rightward
11         }
12     }
13     return ans;
14 }
```

Strict Inequality

Lines 6–7: Checks for strictly greater elements ($\text{arr}[\text{mid}] > \text{target}$).

If $\text{arr}[\text{mid}] \leq \text{target}$, we advance $\text{low} = \text{mid} + 1$ because the target cannot be strictly greater than itself.

Lower and Upper Bound: Duplicate Range Execution

Lower Bound (target = 4)

Array: [1, 2, 4, 4, 4, 6, 7]

- $L = 0, H = 6 \implies \text{mid} = 3, \text{arr}[3] = 4 \geq 4$
 $\text{ans} = 3, H = 2.$
- $L = 0, H = 2 \implies \text{mid} = 1, \text{arr}[1] = 2 < 4$
 $L = 2.$
- $L = 2, H = 2 \implies \text{mid} = 2, \text{arr}[2] = 4 \geq 4$
 $\text{ans} = 2, H = 1.$
- $L > H \implies \text{Lower Bound} = 2.$

Upper Bound (target = 4)

Array: [1, 2, 4, 4, 4, 6, 7]

- $L = 0, H = 6 \implies \text{mid} = 3, \text{arr}[3] = 4 \leq 4$
 $L = 4.$
- $L = 4, H = 6 \implies \text{mid} = 5, \text{arr}[5] = 6 > 4$
 $\text{ans} = 5, H = 4.$
- $L = 4, H = 4 \implies \text{mid} = 4, \text{arr}[4] = 4 \leq 4$
 $L = 5.$
- $L > H \implies \text{Upper Bound} = 5.$

Range: [2..4], Count: $5 - 2 = 3.$

Mental Model: Binary Search on Monotonic Predicates

Beyond Array Indices

Binary Search applies to **any function** $P(x)$ whose boolean evaluation is **monotonic** over a domain $[L, R]$.

- **Monotonic Condition:**
 $x_1 \leq x_2 \implies P(x_1) \leq P(x_2)$ or vice-versa.
- **The Core Paradigm:** If verifying candidate x takes $\mathcal{O}(K)$ time, the entire optimization runs in $\mathcal{O}(K \cdot \log(\text{Range}))$.

Form A: False $\rightarrow$ True (Minimize)



Form B: True $\rightarrow$ False (Maximize)



The Feasibility Function $P(x)$ & Direction Rules

Three Steps of Binary Search on Answer

1. **Define Search Space $[L, R]$:** Determine theoretical minimum and maximum possible answer values.
2. **Design Feasibility Predicate $P(x)$:** Write a helper `isFeasible(x)` returning boolean.
3. **Establish Direction Rule:** If `isFeasible(mid)` is TRUE, can we find a better solution left or right?

Direction Decision Rules

Goal	Feasible ($P(\text{mid}) = \text{T}$)	Action
Min-Max	Record ans high = mid - 1	= mid;
Max-Min	Record ans low = mid + 1	= mid;

$$\text{Total Time} = \mathcal{O}(\text{Cost}(P) \times \log(R - L))$$

Variation 2: Monotonic Math Search — Integer Square Root

Problem Statement

Given integer N , compute $\lfloor \sqrt{N} \rfloor$ (largest integer x such that $x^2 \leq N$) without using built-in math functions.

- **Search Space:** $L = 1, R = N$.
- **Monotonic Predicate:** $P(x) \equiv (x^2 \leq N)$.
- **Transition State:** For $N = 10$, $x \in [1, 2, 3, 4, \dots]$ evaluates to $[T, T, T, F, \dots]$.
- **Goal:** Find the last **TRUE** value in $\mathcal{O}(\log N)$ time.

Integer Overflow Guard

$x \cdot x$ can exceed $2^{31} - 1$ when $x \approx 10^9$. Always compute products using long arithmetic.

Integer Square Root: Java Implementation

```
1 public static int mySqrt(int n) {
2     if (n == 0) return 0;
3     int low = 1, high = n, ans = 1;
4     while (low <= high) {
5         int mid = low + (high - low) / 2;
6         long sq = (long) mid * mid;
7         if (sq <= n) {
8             ans = mid; // Feasible candidate
9             low = mid + 1; // Try larger integer
10        } else {
11            high = mid - 1; // Infeasible: shrink
12        }
13    }
14    return ans;
15 }
```

Algorithm Invariant

Line 6: Cast to (long) mid * mid avoids integer overflow.

Line 8: Because we seek the largest x satisfying $x^2 \leq N$, recording $\text{ans} = \text{mid}$ and testing larger numbers ($\text{low} = \text{mid} + 1$) guarantees finding the exact floor value.

Variation 3: Min-Max Optimization — Book Allocation

Problem Statement

Given N books with pages $arr = [p_1, \dots, p_N]$ and K students, allocate contiguous books such that the **maximum pages assigned to any student is minimized**.

- Every student gets at least one book.
- Allocations must be contiguous subarrays.
- **Greedy on Indices Fails:** Cannot place partitions without global capacity bounds.

Student 1 (Cap ≤ 113)

$[12, 34, 67] \Rightarrow 113$

Student 2 (Cap ≤ 113)

$[90] \Rightarrow 90$

Max Load = $\max(113, 90) = 113$ with $K = 2$

The Optimization Inversion

Do not ask: *"Where should I split the array?"*

Ask: *"Given max capacity M , can K students hold all books?"*

Book Allocation: The Feasibility Predicate $P(x)$

Establishing Search Space

- **Lower Bound (L):** $\max(\text{pages})$ (a student must be able to hold the single largest book).
- **Upper Bound (R):** $\sum \text{pages}$ (one student gets all books).
- **Monotonicity:** If capacity M is feasible with $\leq K$ students, any capacity $M' > M$ is also feasible:

$[F, F, \dots, F, T, T, T, T]$

- **Target:** Find the first TRUE.

Greedy Allocation Logic ($\mathcal{O}(N)$)

1. Current student load = 0, student count = 1.
2. For each book:
 - If current load + page $\leq M$: add to student.
 - Else: allocate new student (count ++), current load = page.
3. Feasible if count $\leq K$.

Book Allocation: Java Implementation

```
1 private static boolean isFeasible(  
2     int[] pages, int k, int maxCap) {  
3     int students = 1, currentLoad = 0;  
4     for (int p : pages) {  
5         if (currentLoad + p <= maxCap) {  
6             currentLoad += p;  
7         } else {  
8             students++;  
9             currentLoad = p;  
10        }  
11    }  
12    return students <= k;  
13 }
```

```
1 public static int findPages(int[] pages, int k) {  
2     if (k > pages.length) return -1;  
3     int low = Arrays.stream(pages).max().getAsInt();  
4     int high = Arrays.stream(pages).sum();  
5     int ans = high;  
6     while (low <= high) {  
7         int mid = low + (high - low) / 2;  
8         if (isFeasible(pages, k, mid)) {  
9             ans = mid; // Try smaller max  
10            high = mid - 1;  
11        } else {  
12            low = mid + 1; // Infeasible: increase  
13        }  
14    }  
15    return ans;  
16 }
```

Book Allocation: Step-by-Step Search Space Reduction

Trace: pages = [12, 34, 67, 90], $K = 2$

Search Space: $L = 90$, $R = 203$.

- **Pass 1:** $L = 90$, $H = 203 \implies \text{mid} = 146$
 $[12, 34, 67] = 113$, $[90] = 90 \implies 2 \leq 2$ (T)
 $\text{ans} = 146$, $H = 145$.
- **Pass 2:** $L = 90$, $H = 145 \implies \text{mid} = 117$
 $[12, 34, 67] = 113$, $[90] = 90 \implies 2 \leq 2$ (T)
 $\text{ans} = 117$, $H = 116$.
- **Pass 3:** $L = 90$, $H = 116 \implies \text{mid} = 103$
Students = $3 > 2$ (F) $\implies L = 104$.
- **Pass 4:**
 $L = 104$, $H = 116 \implies \dots \implies \text{ans} = 113$.

Optimal Student Partition

With optimal minimum capacity **113**:

- **Student 1:** $12 + 34 + 67 = 113 \leq 113$
- **Student 2:** $90 \leq 113$
- **Bottleneck Load:** $\max(113, 90) = 113$

Time Complexity: $\mathcal{O}(N \cdot \log(\sum P))$.

Variation 4: Max-Min Optimization — Aggressive Cows

Problem Statement

Given stall coordinates $\text{stalls} = [x_1, \dots, x_N]$ on a 1D axis and C cows, place cows in stalls such that the **minimum distance between any two cows is maximized**.

- **Search Space:** $L = 1$,
 $R = \text{stalls}[N - 1] - \text{stalls}[0]$.
- **Monotonicity:** If minimum distance D is feasible, any distance $D' < D$ is also feasible:

$[T, T, T, \dots, T, F, F, F]$

- **Target:** Find the last **TRUE**.



Greedy Placement Invariant

Place first cow at $\text{stalls}[0]$. For each subsequent stall, place a cow if $\text{pos} \geq \text{last_pos} + D$.

Aggressive Cows: Java Implementation

```
1 private static boolean canPlace(  
2     int[] stalls, int cows, int dist) {  
3     int count = 1, lastPos = stalls[0];  
4     for (int i = 1; i < stalls.length; i++) {  
5         if (stalls[i] - lastPos >= dist) {  
6             count++;  
7             lastPos = stalls[i];  
8             if (count >= cows) return true;  
9         }  
10    }  
11    return false;  
12 }
```

```
1 public static int maxMinDistance(  
2     int[] stalls, int cows) {  
3     Arrays.sort(stalls); // O(N log N)  
4     int low = 1;  
5     int high = stalls[stalls.length - 1] - stalls[0];  
6     int ans = 1;  
7     while (low <= high) {  
8         int mid = low + (high - low) / 2;  
9         if (canPlace(stalls, cows, mid)) {  
10            ans = mid; // Try larger distance  
11            low = mid + 1;  
12        } else {  
13            high = mid - 1; // Infeasible: shrink  
14        }  
15    }  
16    return ans;  
17 }
```

Aggressive Cows: Visual Placement Verification

Trace: stalls = [1, 2, 4, 8, 9], $C = 3$

Range: $L = 1, R = 9 - 1 = 8$.

- **Check $D = 4$:** Cow 1 at 1, Cow 2 at 1 + 4 ≤ 8 (stall 8), Cow 3 at 8 + 4 ≤ 12 (**No stall!**)
Cows placed = $2 < 3 \implies$ **FALSE** ($H = 3$).
- **Check $D = 3$:** Cow 1 at 1, Cow 2 at 4 ($4 - 1 = 3 \geq 3$), Cow 3 at 8 ($8 - 4 = 4 \geq 3$)
Cows placed = $3 \geq 3 \implies$ **TRUE**
(ans = 3, $L = 4$).
- $L > H \implies$ **Optimal Min-Distance = 3.**

Dual Optimization Duality

- **Min-Max (Capacity):** $P(x)$ is $F \rightarrow T \implies$ First TRUE (high = mid - 1).
- **Max-Min (Distance):** $P(x)$ is $T \rightarrow F \implies$ Last TRUE (low = mid + 1).

Both mirror the exact same binary search skeleton!

Complexity Analysis: Monotonic Binary Search

Time Complexity Derivation

Given search window $W = R - L + 1$ and a verification predicate costing $\mathcal{O}(K)$ time:

$$T(N) = \mathcal{O}(K \cdot \log_2 W)$$

Problem Breakdowns:

- **Element Search:** $\mathcal{O}(\log N)$
- **Integer Sqrt:** $\mathcal{O}(\log N)$
- **Book Allocation:** $\mathcal{O}(N \log(\sum P))$
- **Aggressive Cows:**
 $\mathcal{O}(N \log N + N \log(\text{Range}))$

Space Complexity

- **Auxiliary Memory:** $\mathcal{O}(1)$ in-place iterative pointers.
- **No Dynamic Allocations:** Greedy check runs over the existing array using simple accumulators.

For $N \leq 10^5$ and search range $\leq 10^9$, binary search on answer takes $\approx 30 \times 10^5 = 3 \times 10^6$ operations (≈ 10 ms).

Master Binary Search & Monotonicity Matrix

Problem Archetype	Search $[L, R]$	Space	Predicate $P(x)$	Monotonicity	Optimum
Classic Binary Search	$[0, N - 1]$		$\text{arr}[\text{mid}] == \text{target}$	Discrete	Match
Lower Bound	$[0, N - 1]$		$\text{arr}[\text{mid}] \geq \text{target}$	$F \rightarrow T$	First T
Upper Bound	$[0, N - 1]$		$\text{arr}[\text{mid}] > \text{target}$	$F \rightarrow T$	First T
Integer Square Root	$[1, N]$		$x^2 \leq N$	$T \rightarrow F$	Last T
Book Allocation (Min-Max)	$[\max(P), \sum P]$		$\text{studentsRequired}(M) \leq K$	$F \rightarrow T$	First T
Aggressive Cows (Max-Min)	$[1, x_N - x_1]$		$\text{cowsPlaced}(D) \geq C$	$T \rightarrow F$	Last T

The Universal Template: Every monotonic problem uses the exact same `while (low <= high)` loop. Only the search space $[L, R]$ and predicate $P(x)$ vary across domain requirements.

Idea 4

Two-Pointer Techniques

Converging Invariants, Greedy Elimination & State Space Pruning

Chapter 01: Arrays | Interview Problem Solving Masterclass

The Problem: The 2D State Space Bottleneck

The Pairwise Search Challenge

Many fundamental problems ask us to find, count, or optimize over pairs of indices (i, j) in an array of size N :

- Find two numbers summing to Target.
- Find the two boundaries holding maximum water.
- Find unique triplets summing to zero.

Brute Force Bottleneck: Evaluating all $\binom{N}{2} = \frac{N(N-1)}{2}$ pairs requires $\mathcal{O}(N^2)$ time. For $N = 10^5$, this takes $\approx 5 \times 10^9$ operations $\Rightarrow$ **TLE**.

The Two-Pointer Breakthrough

Can we evaluate or eliminate **multiple candidate pairs simultaneously** in $\mathcal{O}(1)$ time?

The Core Principle: By exploiting sorted order or geometric properties, two independent pointers (L and R) traverse the array such that each comparison safely prunes an **entire subset of impossible candidates**.

Total operations collapse from $\mathcal{O}(N^2) \rightarrow \mathcal{O}(N)$!

The Intuition: “Extreme Helpers” & Safe Elimination

Why Start at Opposite Extremes?

In a sorted array, start with $L = 0$ (smallest) and $R = N - 1$ (largest):

- If **Sum < Target**: $A[R]$ is the **largest possible partner** in the array. If even with $A[R]$ the sum is too small, then $A[L]$ cannot reach target with **any** remaining element ($\leq A[R]$). $A[L]$ is hopeless $\implies$ **Discard L** ($L++$).
- If **Sum > Target**: $A[L]$ is the **smallest possible partner**. If even with $A[L]$ the sum is too big, $A[R]$ cannot reach target with **any** remaining element ($\geq A[L]$). $A[R]$ is hopeless $\implies$ **Discard R** ($R--$).

Step 1: Sum = $2 + 15 = 17 < 18$ (Too Small)



• Max helper 15: $2 + (\leq 15) \leq 17 < 18 \implies$ **Discard 2** ($L++$)

Step 2: Sum = $7 + 15 = 22 > 18$ (Too Large)



• Min helper 7: $15 + (\geq 7) \geq 22 > 18 \implies$ **Discard 15** ($R--$)

Step 3: Sum = $7 + 11 = 18 == 18$ (Exact Match)



• Target Matched at $[1, 2]$! Zero backtracking needed.

Taxonomy of Two-Pointer Strategies

1. Converging

Opposite Ends: $L \rightarrow \leftarrow R$

Pointers start at $[0, N - 1]$ and move towards each other.

- Two Sum (Sorted)
- Container With Water
- Trapping Rain Water

2. Multi-Pointer

Nested Search & Bounds

Fix 1 or more outer pointers; converge inner two pointers.

- 3Sum ($\mathcal{O}(N^2)$)
- 4Sum ($\mathcal{O}(N^3)$)
- 3Sum Closest

3. Fast-Slow

Reader-Writer: $L \rightarrow, R \rightarrow$

Pointers move in same direction at varying rates.

- In-Place Deduplication
- Move Zeroes
- Linked List Cycles

Foundation Problem: Two Sum in a Sorted Array

Problem Statement

Given a **1-indexed** array of integers numbers sorted in non-decreasing order, find two numbers such that they add up to a target sum. Return their 1-based indices $[i + 1, j + 1]$.

Constraint: Exactly one solution exists; cannot use the same element twice.

Algorithmic Trade-offs

- **Brute Force:** $\mathcal{O}(N^2)$ time, $\mathcal{O}(1)$ space.
- **Hash Map:** $\mathcal{O}(N)$ time, $\mathcal{O}(N)$ space (Ignores sorted order; allocates heap memory).
- **Two Pointers:** $\mathcal{O}(N)$ time, $\mathcal{O}(1)$ space (Optimal! Fully exploits sorted order with zero memory allocations).

Two Sum (Sorted): Java Implementation

```
1 public static int[] twoSum(int[] numbers, int target) {
2     int left = 0, right = numbers.length - 1;
3     while (left < right) {
4         int sum = numbers[left] + numbers[right];
5         if (sum == target) {
6             return new int[]{left + 1, right + 1};
7         } else if (sum < target) {
8             left++;    // Discard left: sum too small
9         } else {
10            right--;   // Discard right: sum too large
11        }
12    }
13    return new int[]{-1, -1};
14 }
```

Line-by-Line Mechanics

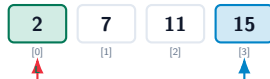
- **Line 2:** Initializes two boundary pointers at opposite extremes ($L = 0, R = N - 1$).
- **Line 5:** Calculates pair sum in $\mathcal{O}(1)$ time.
- **Line 8:** If $\text{sum} < \text{target}$, incrementing L increases the sum.
- **Line 10:** If $\text{sum} > \text{target}$, decrementing R decreases the sum.

Two Sum: Step-by-Step State Evolution

Trace: numbers = [2, 7, 11, 15], $T = 18$

- **Pass 1:** $L = 0(2), R = 3(15)$
Sum = $2 + 15 = 17 < 18$
 $\implies L$ advances to 1.
- **Pass 2:** $L = 1(7), R = 3(15)$
Sum = $7 + 15 = 22 > 18$
 $\implies R$ decreases to 2.
- **Pass 3:** $L = 1(7), R = 2(11)$
Sum = $7 + 11 = 18 == 18$
 $\implies$ **Match Found!** Return [2, 3].

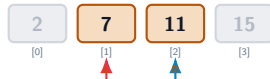
Pass 1: Sum = $2 + 15 = 17 < 18 \implies L + +$



Pass 2: Sum = $7 + 15 = 22 > 18 \implies R - -$



Pass 3: Sum = $7 + 11 = 18 == 18 \implies$ Match [2, 3]!



Invariant Analysis: Why No Valid Pair is Missed

Formal Correctness Invariant

Loop Invariant: If a valid pair (x, y) summing to Target exists in the array, then at the start of every while-loop iteration:

$$L \leq x < y \leq R$$

Proof by Induction:

- **Base:** $L = 0, R = N - 1$ spans the entire array.
- **Step on $L++$:** Since $A[L] + A[R] < T$, for all $k \leq R$, $A[L] + A[k] \leq A[L] + A[R] < T$. $A[L]$ cannot pair with any remaining element. Discarding L is safe.

Step on $R--$

Since $A[L] + A[R] > T$, for all $k \geq L$:

$$A[k] + A[R] \geq A[L] + A[R] > T$$

Element $A[R]$ cannot pair with any element remaining in the active window. Discarding R loses zero solutions.

Termination: Either a match is returned or $L \geq R$, proving no valid pair exists.

Variation 1: Multi-Pointer Search — 3Sum

Problem Statement

Given an integer array `nums`, return all unique triplets $[A[i], A[j], A[k]]$ such that $i \neq j \neq k$ and:

$$A[i] + A[j] + A[k] = 0$$

Brute Force: 3 nested loops take $\mathcal{O}(N^3)$ time.

The 2-Pointer Reduction:

1. Sort `nums` in $\mathcal{O}(N \log N)$.
2. Fix index i . Find pairs in $[i + 1, N - 1]$ summing to $-A[i]$.
3. Total Time: $\mathcal{O}(N^2)$.

Our First Strategy

Let's first write the straightforward code by plugging the standard Two-Pointer algorithm inside a for-loop.

Question: Will this naive solution directly solve the problem, or does a subtle trap emerge when the input array contains duplicate numbers?

Let's inspect the initial code implementation first!

3Sum: First Attempt (Without Deduplication)

```
1 public static List<List<Integer>> threeSumNaive(int[] a) {
2     Arrays.sort(a);
3     List<List<Integer>> res = new ArrayList<>();
4     for (int i = 0; i < a.length - 2; i++) {
5         int L = i + 1, R = a.length - 1;
6         while (L < R) {
7             int sum = a[i] + a[L] + a[R];
8             if (sum == 0) {
9                 res.add(List.of(a[i], a[L], a[R]));
10                L++; R--; // Naive pointer movement
11            } else if (sum < 0) {
12                L++;
13            } else {
14                R--;
15            }
16        }
17    }
18    return res;
19 }
```

The Naive Logic

- Line 4: Loops outer anchor i from 0 to $N - 3$.
- Line 8: Records matched triplet $[a[i], a[L], a[R]]$.
- Line 9: Advances both $L++$ and $R--$.

What goes wrong? When the array contains duplicate numbers, this code generates **duplicate triplets** in the output list!

The Issue: Duplicate Triplet Explosion

Why Duplicates Occur

Consider sorted nums = $[-1, -1, 0, 1, 1]$:

- At $i = 0$ (val = -1): Finds $L = 2(0), R = 3(1) \Rightarrow [-1, 0, 1]$.
Next $R = 4(1) \Rightarrow [-1, 0, 1]$ (Duplicate!)
- At $i = 1$ (val = -1): Same anchor value -1 repeats the entire search $\Rightarrow [-1, 0, 1]$ (Duplicate Again!)

Output: $[-1, 0, 1], [-1, 0, 1], [-1, 0, 1] \Rightarrow$
WRONG!



- Match 1: $A[0] + A[2] + A[3] = [-1, 0, 1]$ (First Encountered)
- Duplicate 1: Moving R to index 4 gives $A[0] + A[2] + A[4] = [-1, 0, 1]$ (DUP!)
- Duplicate 2: Advancing to $i = 1$ ($A[1] = -1$) re-scans identical $[-1, 0, 1]$ (DUP!)

Why Not Use HashSet?: Wrapping output in a HashSet costs $\mathcal{O}(N)$ heap memory and hashing overhead. We must deduplicate **via pointer skipping** in $\mathcal{O}(1)$ space!

The Solution: The Two Deduplication Gates

1. Outer Anchor Deduplication

When fixing pointer i , skip duplicate values:

```
if (i > 0 && nums[i] == nums[i - 1]) {  
    continue; // Skip duplicate anchor  
}
```

Crucial Nuance: Check against $i - 1$, **never** $i + 1$.
Checking $i + 1$ would wrongly discard valid triplets like $[-1, -1, 2]$ where $i = 0$ and $L = 1$ share identical values.

2. Inner Pointer Deduplication

After recording a match ($A[i] + A[L] + A[R] == 0$):

```
while (L < R && nums[L] == nums[L + 1]) L++;  
while (L < R && nums[R] == nums[R - 1]) R-;  
L++; R-;
```

Why: Compress pointers past all identical neighbors so the next evaluated pair is strictly distinct.

3Sum: Correct Java Implementation

```
1 public static List<List<Integer>> threeSum(int[] a) {
2     Arrays.sort(a);
3     List<List<Integer>> res = new ArrayList<>();
4     for (int i = 0; i < a.length - 2; i++) {
5         if (i > 0 && a[i] == a[i - 1]) continue;
6         int L = i + 1, R = a.length - 1;
7         while (L < R) {
8             int sum = a[i] + a[L] + a[R];
9             if (sum == 0) {
10                 res.add(List.of(a[i], a[L], a[R]));
11                 while (L < R && a[L] == a[L + 1]) L++;
12                 while (L < R && a[R] == a[R - 1]) R--;
13                 L++; R--;
14             } else if (sum < 0) L++;
15             else R--;
16         }
17     }
18     return res;
19 }
```

Key Highlights

- **Line 4:** Gate 1 skips duplicate outer anchors in $\mathcal{O}(1)$.
- **Line 8:** Direct three-sum evaluation against zero.
- **Lines 11–12:** Gate 2 compresses pointers past identical keys.
- **Complexity:** $\mathcal{O}(N^2)$ time, $\mathcal{O}(1)$ auxiliary space.

3Sum: Step-by-Step Triplet Extraction Trace

Trace: `nums = [-4, -1, -1, 0, 1, 2]`

- $i = 0(\text{val} = -4)$: Target sum = 4.
 $L=1, R=5 \implies -1 + 2 = 1 < 4 \implies$ No match.
- $i = 1(\text{val} = -1)$: Target sum = 1.
 - $L=2, R=5 \implies -1 + 2 = 1 \implies$ **Triplet** `[-1, -1, 2]`! Skip dups.
 - $L=3, R=4 \implies 0 + 1 = 1 \implies$ **Triplet** `[-1, 0, 1]`!
- $i = 2(\text{val} = -1)$: `nums[2] == nums[1]  $\implies$  SKIPPED (Gate 1)!`
- $i = 3(0), i = 4(1)$: No further matches.

Extracted Result

`[[-1, -1, 2], [-1, 0, 1]]`

- Zero duplicate triplets emitted.
- No HashSets or extra data structures allocated.
- Linear scans inside outer loop total exactly $O(N)$ per outer iteration.

Variation 2: Greedy Elimination — Container With Water

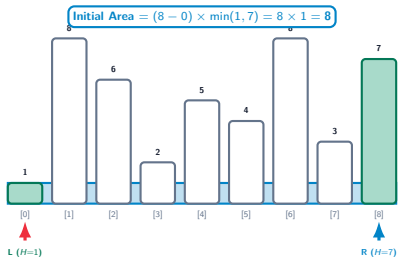
Problem Formulation

Given N vertical lines $H[0 \dots N-1]$, find two lines that form a container holding the maximum water.

Capacity Formula:

$$\text{Area}(L, R) = (R - L) \times \min(H[L], H[R])$$

Start: Maximum width $L = 0, R = N - 1$.



The Pointer Dilemma: Moving any pointer inward shrinks width from $(R - L)$ to $(R - L - 1)$. To compensate, height **must increase**. Which pointer to shift: L or R ?

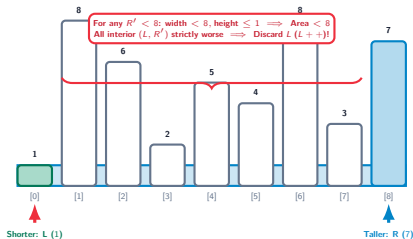
Container With Most Water: The Elimination Proof

The Bottleneck Principle

Suppose $H[L] < H[R]$. The container height is limited strictly by shorter bar $H[L]$.

Pairing L with any interior bar $R' < R$:

- Width is strictly smaller: $R' - L < R - L$.
- Height cannot exceed $H[L]$:
 $\min(H[L], H[R']) \leq H[L]$.
- Thus: $\text{Area}(L, R') < \text{Area}(L, R)$.



Universal Invariant: Always advance the pointer pointing to the **shorter** vertical bar:

if $H[L] < H[R] \implies L++$; else $R--$;

Container With Most Water: Java Implementation

```
1 public static int maxArea(int[] h) {
2     int left = 0, right = h.length - 1;
3     int maxWater = 0;
4     while (left < right) {
5         int width = right - left;
6         int currentWater =
7             width * Math.min(h[left], h[right]);
8         maxWater = Math.max(maxWater, currentWater);
9         if (h[left] < h[right]) {
10             left++; // Discard shorter left
11         } else {
12             right--; // Discard shorter right
13         }
14     }
15     return maxWater;
16 }
```

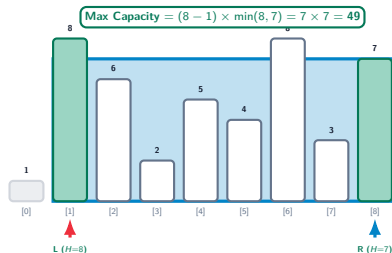
Execution Complexity

- **Time Complexity:** $\mathcal{O}(N)$
Each step moves either L right or R left, terminating in exactly $N - 1$ steps.
- **Space Complexity:** $\mathcal{O}(1)$
Constant auxiliary variables with zero memory overhead.

Container With Most Water: Visual Capacity Trace

Trace: [1, 8, 6, 2, 5, 4, 8, 3, 7]

- ($L=0, R=8$): Area = $8 \times 1 = 8 \implies L++$.
- ($L=1, R=8$): Area = $7 \times 7 = 49 \implies R--$.
- ($L=1, R=7$): Area = $6 \times 3 = 18 \implies R--$.
- ($L=1, R=6$): Area = $5 \times 8 = 40 \implies R--$.
- ($L=1, R=5$): Area = $4 \times 4 = 16 \implies R--$.
- **Max Capacity = 49** at lines (1, 8).



Global Optimality: We evaluated only 8 configurations out of $\binom{9}{2} = 36$ possibilities, yet mathematical pruning guarantees the global maximum was found.

Variation 3: Same-Direction — Fast-Slow Pointers

Problem Statement

Given a sorted array `nums`, remove duplicate elements **in-place** such that each unique element appears once. Return the new valid length K .

Constraints: Must modify array in-place with $\mathcal{O}(1)$ extra memory.

Reader-Writer Pointer Roles

- **Slow Pointer (slow):** The **Writer**. Demarcates the boundary of the unique prefix array $[0 \dots \text{slow}]$.
- **Fast Pointer (fast):** The **Reader / Explorer**. Scans every element from index 1 to $N - 1$.
- **Invariant:** When `nums[fast]  $\neq$  nums[slow]`, advance `slow ++` and copy `nums[slow] = nums[fast]`.

In-Place Deduplication: Java Implementation

```
1 // Unique Elements Only
2 public static int removeDuplicates(int[] nums) {
3     if (nums.length == 0) return 0;
4     int slow = 0;
5     for (int fast = 1; fast < nums.length; fast++) {
6         if (nums[fast] != nums[slow]) {
7             slow++;
8             nums[slow] = nums[fast];
9         }
10    }
11    return slow + 1;
12 }
```

```
1 // Generalization: Allow at most K duplicates
2 public static int removeDuplicatesK(int[] nums, int k) {
3     int slow = 0;
4     for (int fast = 0; fast < nums.length; fast++) {
5         if (slow < k || nums[fast] != nums[slow - k]) {
6             nums[slow++] = nums[fast];
7         }
8     }
9     return slow;
10 }
```

Fast-Slow Pointer: Visual Memory Compaction

Trace: `nums = [1, 1, 2, 2, 3]`

- Start: `slow = 0, fast = 1` $\Rightarrow$ Equal $\Rightarrow$ `fast++`.
- `fast = 2(2) \neq A[0]` $\Rightarrow$ `slow = 1, A[1] = 2`.
- `fast = 3(2) == A[1]` $\Rightarrow$ Equal $\Rightarrow$ `fast++`.
- `fast = 4(3) \neq A[1]` $\Rightarrow$ `slow = 2, A[2] = 3`.
- Compacted Prefix:** `[1, 2, 3]`, `length = 3`.



Core Power: Fast-Slow pointers perform in-place array compaction in a single $\mathcal{O}(N)$ scan without extra buffers.

The Boundary: Two Pointers vs. Sliding Window

Two Pointers (Idea 4)

- **Focus:** Point-Pairs ($A[L]$, $A[R]$)
- **Action:** Eliminate 2D search space via bounds
- **State:** Interior elements ignored
- **Ex:** Two Sum, 3Sum, Container With Water

Sliding Window (Idea 5)

- **Focus:** Contiguous Subarrays $A[L..R]$
- **Action:** Incremental state maintenance (add/drop)
- **State:** Aggregate (Sum, Frequency map)
- **Ex:** Min Subarray Sum, Longest Substring

The Distinction Invariant

- **Two Pointers:** We care **only** about the values at indices L and R . Elements between them are skipped/pruned.
- **Sliding Window (Idea 5):** We care about **every contiguous element** in range $[L..R]$ (e.g. subarray sum, char frequencies).

Complexity Analysis: Two-Pointer Paradigms

Time Complexity Derivations

- **Two Sum (Sorted):** $\mathcal{O}(N)$ pointer convergence.
- **3Sum:** $\mathcal{O}(N \log N)$ sort + $\mathcal{O}(N \times N)$ nested two pointers = $\mathcal{O}(N^2)$.
- **Container With Water:** $\mathcal{O}(N)$ single pass.
- **In-Place Deduplication:** $\mathcal{O}(N)$ fast pointer scan.

Space Complexity

- **Auxiliary Memory:** $\mathcal{O}(1)$ in-place pointers across all variations.
- **No Dynamic Allocations:** HashSets and auxiliary arrays are completely eliminated by leveraging sorted invariants.

Ideal for memory-constrained competitive environments.

Master Two-Pointer Pattern Matrix

Archetype	Topology	Elimination Invariant	Time	Space
Two Sum	Converging ($L \rightarrow \leftarrow R$)	$\text{Sum} < T \implies L++$; $\text{Sum} > T \implies R--$	$\mathcal{O}(N)$	$\mathcal{O}(1)$
3Sum	Outer $i + (L \rightarrow \leftarrow R)$	Target = $-A[i]$; Skip duplicate neighbors	$\mathcal{O}(N^2)$	$\mathcal{O}(1)$
Container	Converging ($L \rightarrow \leftarrow R$)	Discard shorter line: $\min(H[L], H[R])$	$\mathcal{O}(N)$	$\mathcal{O}(1)$
Deduplication	Fast-Slow ($L \rightarrow, R \rightarrow$)	If $A[\text{fast}] \neq A[\text{slow}]$, advance $\text{slow}++$	$\mathcal{O}(N)$	$\mathcal{O}(1)$

Universal Principle: Two pointers is fundamentally about **safe search-space elimination**. Monotonicity or geometric bottlenecks allow pruning vast candidate spaces in $\mathcal{O}(1)$ per step.

Idea 5

Sliding Window Technique

Fixed vs. Variable Windows, Invariant Expansion/Contraction & Monotonic Deques

Contiguous Subarrays: The Redundancy Trap

Naive Recomputation ($\mathcal{O}(N \cdot K)$)

Given array A and window size K :

- Window 1: $A[0] + A[1] + \dots + A[K - 1]$ (K ops).
- Window 2: $A[1] + A[2] + \dots + A[K]$ (K ops).
- Window 3: $A[2] + A[3] + \dots + A[K + 1]$ (K ops).

Flaw: Recalculates the entire window from scratch every step!

Total Cost: $\mathcal{O}(N \cdot K) \Rightarrow$ **TLE** on large K !

The Sliding Window Insight

Adjacent windows share $K - 1$ elements in common!

$$\text{NextState} = \text{CurrentState} + A[R] - A[L]$$

- **Ingest:** Add incoming element $A[R]$ on right ($\mathcal{O}(1)$).
- **Evict:** Drop outgoing element $A[L]$ on left ($\mathcal{O}(1)$).
- **Amortized Cost:** $\mathcal{O}(N)$ total linear time!

Taxonomy: Fixed vs. Variable Windows

Archetype 1: Fixed Window Size

- **Given:** Window length K is fixed in the input.
- **State:** Running aggregate over exactly K contiguous elements.
- **Movement:** Both L and R advance together ($L++$, $R++$) once window reaches size K .
- **Examples:** Max Sum Subarray of size K , First Negative in Window, Sliding Window Maximum, Count Anagrams.

Archetype 2: Variable Window Size

- **Given:** Condition / Target K (e.g., Sum = K , K distinct chars).
- **State:** Dynamic window $[L \dots R]$ that expands and contracts.
- **Movement:** Expand R to satisfy condition; shrink L when condition is violated.
- **Examples:** Longest Subarray with Sum K , At Most K Distinct Chars, Longest Unique Substring, Min Window Substring.

Canonical Template 1: Fixed-Size Window Engine

```
1 int L = 0;
2 for (int R = 0; R < N; R++) {
3     // 1. Ingest incoming element
4     state.add(A[R]);
5
6     // 2. Wait until window reaches size K
7     if (R - L + 1 == K) {
8         // 3. Record / update answer
9         ans = update(ans, state);
10
11        // 4. Slide window: evict leftmost element
12        state.remove(A[L]);
13        L++;
14    }
15 }
```

The 4-Step Rhythm

- **Step 1 (Ingest):** Add $A[R]$ to running state.
- **Step 2 (Check Size):** Window is active when span $R - L + 1 == K$.
- **Step 3 (Harvest):** Extract answer from active window.
- **Step 4 (Evict & Slide):** Remove $A[L]$ and increment $L++$ to maintain span K .

Foundational Problem 1: Maximum Sum Subarray of Size K

```
1 public static long maxSubarraySum(int[] arr, int k) {
2     long currentSum = 0, maxSum = Long.MIN_VALUE;
3     int L = 0;
4     for (int R = 0; R < arr.length; R++) {
5         currentSum += arr[R]; // Ingest
6         if (R - L + 1 == k) {
7             maxSum = Math.max(maxSum, currentSum);
8             currentSum -= arr[L]; // Evict
9             L++;
10        }
11    }
12    return maxSum;
13 }
```

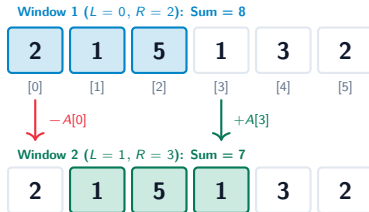
Key Highlights

- **Line 5:** Ingests incoming element $A[R]$ in $\mathcal{O}(1)$ time.
- **Line 6:** Triggers once window reaches exactly size K .
- **Line 8:** Evicts outgoing element $A[L]$ in $\mathcal{O}(1)$ time.
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(1)$ space.

Maximum Sum Subarray: Visual Window Walkthrough

Trace: $A = [2, 1, 5, 1, 3, 2]$, $K = 3$

- **Window 1** $[0 \dots 2]$: $[2, 1, 5] \implies \text{Sum} = 8$.
Max = 8.
- **Slide to** $[1 \dots 3]$:
 - Subtract $A[0] = 2$, Add $A[3] = 1$.
 - Sum = $8 - 2 + 1 = 7$.
- **Slide to** $[2 \dots 4]$:
 - Subtract $A[1] = 1$, Add $A[4] = 3$.
 - Sum = $7 - 1 + 3 = 9$. Max = 9.
- **Final Maximum Sum: 9** (subarray $[5, 1, 3]$).



Takeaway: Exactly 1 addition and 1 subtraction per step replaces a K -element loop!

Variation 1: First Negative in Window of Size K

Problem Statement

Given array A of size N and integer K , for each contiguous window of size K , find the **first negative integer** appearing in that window.

If a window contains no negative integers, record 0.

Input: $A = [12, -1, -7, 8, -15, 30]$, $K = 3$.

The Queue State Tracker

- Maintain a FIFO 'Queue<Integer>' storing only the negative numbers in the current window.
- **Ingest:** If $A[R] < 0$, add to queue tail.
- **Answer:** 'queue.isEmpty() ? 0 : queue.peek()'.
- **Evict:** If outgoing $A[L] == \text{queue.peek}()$, poll it!

First Negative in Window: Java Implementation

```
1 public static long[] firstNegative(long[] a, int n, int k) {
2     long[] res = new long[n - k + 1];
3     Queue<Long> q = new LinkedList<>();
4     int L = 0, idx = 0;
5     for (int R = 0; R < n; R++) {
6         if (a[R] < 0) q.add(a[R]); // Ingest
7         if (R - L + 1 == k) {
8             res[idx++] = q.isEmpty() ? 0 : q.peek();
9             if (!q.isEmpty() && a[L] == q.peek()) {
10                 q.poll(); // Evict
11             }
12             L++;
13         }
14     }
15     return res;
16 }
```

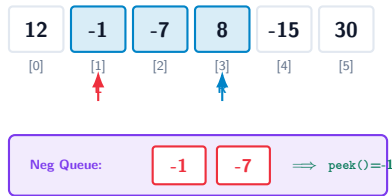
Key Highlights

- **Line 6:** Only negative values enter the queue.
- **Line 8:** 'q.peek()' retrieves the earliest negative in $O(1)$.
- **Line 9:** Discards elements as they fall out of window.
- **Complexity:** $O(N)$ time, $O(K)$ space.

First Negative in Window: Visual Trace

Trace: $A = [12, -1, -7, 8, -15, 30]$, $K = 3$

- **Window 1** $[0 \dots 2]$:
 $[12, -1, -7] \Rightarrow Q = [-1, -7]$.
Ans: -1 .
- **Window 2** $[1 \dots 3]$:
 $[-1, -7, 8] \Rightarrow Q = [-1, -7]$.
Ans: -1 . ($A[0] = 12$ wasn't in Q).
- **Window 3** $[2 \dots 4]$:
 $[-7, 8, -15] \Rightarrow A[1] = -1$ polled!
 $Q = [-7, -15] \Rightarrow$ Ans: -7 .
- **Window 4** $[3 \dots 5]$:
 $[8, -15, 30] \Rightarrow A[2] = -7$ polled!
 $Q = [-15] \Rightarrow$ Ans: -15 .



Result Array: $[-1, -1, -7, -15]$. Every query answered in $\mathcal{O}(1)$!

Variation 2: Sliding Window Maximum

Problem Statement (LeetCode 239)

Given array A and window size K , find the maximum element in **every** sliding window of size K .

The Trap: Why Max-Heap is Suboptimal!

- A Max-Heap can find max in $\mathcal{O}(1)$, but heap deletion of arbitrary outgoing element takes $\mathcal{O}(K)$.
- Total Heap Time: $\mathcal{O}(N \log K)$.
- Can we achieve optimal $\mathcal{O}(N)$? **Yes**, via **Monotonic Deque**!

The Monotonic Deque Invariant

Maintain a double-ended queue ('Deque') of indices with strictly **decreasing** values:

- **Front:** Always holds the index of the **maximum** element in the window.
- **Eviction of Inferior Elements:** If incoming $A[R] \geq A[\text{back}]$, the back element is older and smaller $\implies$ it can **never** be maximum. Pop it!

Sliding Window Maximum: Java Implementation

```
1 public static int[] maxSlidingWindow(int[] a, int k) {
2     int n = a.length, L = 0, idx = 0;
3     int[] res = new int[n - k + 1];
4     Deque<Integer> dq = new ArrayDeque<>();
5     for (int R = 0; R < n; R++) {
6         // Evict smaller elements from back
7         while (!dq.isEmpty() &&
8             a[dq.peekLast()] <= a[R]) {
9             dq.pollLast();
10        }
11        dq.addLast(R);
12        if (R - L + 1 == k) {
13            res[idx++] = a[dq.peekFirst()];
14            if (dq.peekFirst() == L) dq.pollFirst();
15            L++;
16        }
17    }
18    return res;
19 }
```

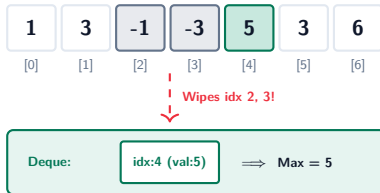
Key Highlights

- **Lines 7–10:** Maintains strictly decreasing invariant in $\mathcal{O}(1)$ amortized.
- **Line 13:** Window max retrieved from 'peekFirst()' in $\mathcal{O}(1)$.
- **Line 14:** Evicts index L as it leaves window.
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(K)$ space.

Sliding Window Maximum: Visual Walkthrough

Trace: $A = [1, 3, -1, -3, 5, 3, 6]$, $K = 3$

- $R = 0$ (1): Deque $[0(1)]$.
- $R = 1$ (3): $3 \geq 1 \implies \text{pop } 0$. Deque $[1(3)]$.
- $R = 2$ (-1): Deque $[1(3), 2(-1)]$. Max = 3.
- $R = 3$ (-3): $L = 0$ leaves. Deque $[1(3), 2(-1), 3(-3)]$. Max = 3.
- $R = 4$ (5): 5 evicts 3, -1, -3! Deque $[4(5)]$. Max = 5.
- $R = 5$ (3): Deque $[4(5), 5(3)]$. Max = 5.



Amortized Proof: Each element enters the deque once and is popped at most once $\implies \leq 2N$ deque operations!

Variation 3: Count Occurrences of Anagrams

Problem Statement (LeetCode 438)

Given text string S and pattern string P , count the total number of start indices of all substring anagrams of P in S .

Definition: An anagram is a permutation of letters with identical character frequencies.

Fixed Window Size: $K = |P|$.

Frequency Match Invariant

- Precompute pattern frequency: 'freqP[26]'.
- Maintain window frequency: 'freqW[26]'.
- Track 'count': number of distinct characters with matching frequencies.
- If 'count == totalDistinctChars', increment anagram count!

Count Anagrams: Java Implementation

```
1 public static int countAnagrams(String s, String p) {
2     int[] pCount = new int[26], wCount = new int[26];
3     for (char c : p.toCharArray()) pCount[c - 'a']++;
4     int k = p.length(), L = 0, matches = 0;
5     for (int R = 0; R < s.length(); R++) {
6         wCount[s.charAt(R) - 'a']++; // Ingest
7         if (R - L + 1 == k) {
8             if (Arrays.equals(pCount, wCount)) matches++;
9             wCount[s.charAt(L) - 'a']--; // Evict
10            L++;
11        }
12    }
13    return matches;
14 }
```

Key Highlights

- **Line 6:** Ingests incoming character count in $\mathcal{O}(1)$.
- **Line 8:** 'Arrays.equals' on 26 integers takes $\mathcal{O}(26) = \mathcal{O}(1)$ time!
- **Line 9:** Evicts outgoing character count at L .
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(1)$ space.

Canonical Template 2: Variable-Size Window Engine

```
1 int L = 0;
2 for (int R = 0; R < N; R++) {
3     // 1. Ingest incoming element (Expand)
4     state.add(A[R]);
5
6     // 2. Contract while window is invalid
7     while (isInvalid(state)) {
8         state.remove(A[L]);
9         L++;
10    }
11
12    // 3. Window is now valid: update answer
13    ans = update(ans, R - L + 1);
14 }
```

The Expand-Shrink Rhythm

- **Step 1 (Expand R):** Greedily ingest $A[R]$ into state.
- **Step 2 (Shrink L):** While constraint is violated, shrink from left until valid again.
- **Step 3 (Harvest):** Active window $[L \dots R]$ is now guaranteed valid; update best length.

Foundational Problem 2: Longest Subarray with Sum K

```
1 public static int longestSubarraySumK(int[] a, long k) {
2     long currentSum = 0;
3     int maxLen = 0, L = 0;
4     for (int R = 0; R < a.length; R++) {
5         currentSum += a[R]; // Expand
6         while (currentSum > k && L <= R) {
7             currentSum -= a[L]; // Shrink
8             L++;
9         }
10        if (currentSum == k) {
11            maxLen = Math.max(maxLen, R - L + 1);
12        }
13    }
14    return maxLen;
15 }
```

Key Highlights

- **Condition:** Non-negative numbers guarantee sum monotonicity.
- **Lines 6–9:** If 'currentSum > k', shrinking L is guaranteed to reduce sum.
- **Line 10:** Records longest valid span $R - L + 1$.
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(1)$ space.

Variation 1: At Most K Distinct Characters

Problem Statement (LeetCode 340)

Given a string S and an integer K , find the length of the **longest substring** that contains at most K **distinct** characters.

Example: $S = \text{"eceba"}, K = 2 \implies \text{"ece"}$ (length 3).

Dynamic Window Invariant

- Maintain `'Map<Character, Integer>'` tracking frequencies in current window.
- **Violation:** `'map.size() > K'`.
- **Shrink:** Decrement frequency of $S[L]$. When count hits 0, `'map.remove(S[L])'` and $L++$.
- **Harvest:** `'maxLen = Math.max(maxLen, R - L + 1)'`.

At Most K Distinct: Java Implementation

```
1 public static int lengthOfLongestSubstringKDistinct(  
2     String s, int k) {  
3     Map<Character, Integer> map = new HashMap<>();  
4     int maxLen = 0, L = 0;  
5     for (int R = 0; R < s.length(); R++) {  
6         char in = s.charAt(R);  
7         map.put(in, map.getOrDefault(in, 0) + 1);  
8         while (map.size() > k) {  
9             char out = s.charAt(L);  
10            map.put(out, map.get(out) - 1);  
11            if (map.get(out) == 0) map.remove(out);  
12            L++;  
13        }  
14        maxLen = Math.max(maxLen, R - L + 1);  
15    }  
16    return maxLen;  
17 }
```

Key Highlights

- **Line 6:** Increments frequency of incoming character $S[R]$.
- **Lines 8–13:** Shrinks L until distinct character count $\leq K$.
- **Line 14:** Harvests longest valid span in $\mathcal{O}(1)$.
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(K)$ space.

Variation 2: Longest Substring Without Repeating

Problem Statement (LeetCode 3)

Given a string S , find the length of the **longest substring** without repeating characters.

Example: $S = \text{"pwwkew"} \implies \text{"wke"}$ (length 3).

Special Case: $K = \text{Unique}$ (every character in window has frequency exactly 1).

Contraction Invariant

- Track visited characters in 'boolean[128]' or frequency map.
- If incoming character $S[R]$ is already present in window $\implies$ **Violation!**
- Shrink L and remove $S[L]$ until the previous instance of $S[R]$ is evicted!

No Repeating Characters: Java Implementation

```
1 public static int lengthOfLongestSubstring(String s) {
2     boolean[] visited = new boolean[128];
3     int maxLen = 0, L = 0;
4     for (int R = 0; R < s.length(); R++) {
5         char in = s.charAt(R);
6         while (visited[in]) {
7             visited[s.charAt(L)] = false;
8             L++;
9         }
10        visited[in] = true;
11        maxLen = Math.max(maxLen, R - L + 1);
12    }
13    return maxLen;
14 }
```

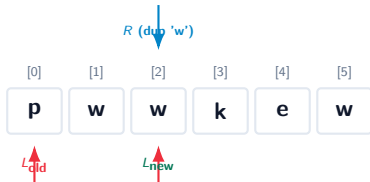
Key Highlights

- **Line 2:** Direct ASCII lookup array gives cache-optimal $\mathcal{O}(1)$ checks.
- **Lines 6–9:** Evicts characters from left until duplicate is cleared.
- **Line 11:** Guaranteed no duplicates $\implies$ valid span.
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(1)$ space.

No Repeating Characters: Visual Trace

Trace: $S = \text{"pwwkew"}$

- $R = 0$ ('p'): Window "p", len = 1.
- $R = 1$ ('w'): Window "pw", len = 2.
- $R = 2$ ('w'): **Duplicate 'w'!**
 - Evict 'p' ($L = 1$), evict 'w' ($L = 2$).
 - Window "w", len = 1.
- $R = 3$ ('k'): Window "wk", len = 2.
- $R = 4$ ('e'): Window "wke", len = 3 (**Max!**).
- $R = 5$ ('w'): Evict 'w' ($L = 3$). Window "kew", len = 3.



- Dup 'w': shrink L past idx 1
- New valid window: [2 . . . 2] ("w")

Result: Max unique substring is "wke" with length 3.

Variation 3: Minimum Window Substring

Problem Statement (LeetCode 76)

Given strings S and T , return the **minimum window substring** of S such that every character in T (including duplicates) is included.

Example: $T = \text{"SAGAR"}$ contains 1 'S', 2 'A', 1 'G', 1 'R'.

Find the shortest substring in S satisfying all required character counts.

Dual-Phase Invariant

- **Phase 1 (Expand R to Satisfy):** Advance R until window contains all required character frequencies.
- **Phase 2 (Contract L to Minimize):** While window remains valid, update minimum answer and shrink L to eliminate excess prefix characters.

Minimum Window: Frequency Balancing Mechanics

The Match Counter

Precompute target counts in array 'map[128]':

- Maintain 'count = T.length()':
- When expanding R : if 'map[S[R]] > 0', decrement 'count--'. Always decrement 'map[S[R]]--'.
- When 'count == 0': Window contains all characters of T !

Target: "SAGAR" $\Rightarrow$ {S : 1, A : 2, G : 1, R : 1}



Matched 5/5 chars $\Rightarrow$ Shrink L to minimize!

Contraction Rule: If 'map[S[L]] < 0', character $S[L]$ is an unnecessary excess character. Shrink safely!

Minimum Window Substring: Java Implementation

```
1 public static String minWindow(String s, String t) {
2     int[] map = new int[128];
3     for (char c : t.toCharArray()) map[c]++;
4     int count = t.length(), L = 0;
5     int minLen = Integer.MAX_VALUE, start = 0;
6     for (int R = 0; R < s.length(); R++) {
7         if (map[s.charAt(R)]-- > 0) count--;
8         while (count == 0) { // Valid!
9             if (R - L + 1 < minLen) {
10                 minLen = R - L + 1;
11                 start = L;
12             }
13             if (++map[s.charAt(L)] > 0) count++;
14             L++;
15         }
16     }
17     return minLen == Integer.MAX_VALUE ? ""
18         : s.substring(start, start + minLen);
19 }
```

Key Highlights

- **Line 7:** Ingests incoming char; decrements 'count' only for required chars.
- **Line 8:** While window is valid, minimizes L .
- **Line 13:** Re-increments map; invalidates when required char dropped.
- **Complexity:** $\mathcal{O}(N + M)$ time, $\mathcal{O}(1)$ space.

Amortized Complexity: Why Sliding Window is $\mathcal{O}(N)$

The Nested While-Loop Fallacy

- Beginners look at the nested 'while (condition)' loop inside the 'for ($R = 0 \dots N$)' loop and assume $\mathcal{O}(N^2)$.
- **The Mathematical Reality:**
 - Pointer R advances from $0 \rightarrow N - 1$ (exactly N increments).
 - Pointer L advances from $0 \rightarrow N - 1$ (at most N increments).
 - L **never moves backward!**
- **Total Pointer Steps:** $\leq 2N$.

Amortized Invariant

Each array element is visited:

1. **Once** when ingested by right pointer R .
2. **At most once** when evicted by left pointer L .

$$\text{Total Work} = \mathcal{O}(N) + \mathcal{O}(N) = \mathcal{O}(N)$$

Constant auxiliary space $\mathcal{O}(1)$ or $\mathcal{O}(K)$ for hash map / frequency array.

Master Sliding Window Pattern Matrix

Archetype	Type	State Tracker & Invariant	Time	Space
Max Sum K	Fixed	Ingest $A[R]$, evict $A[L]$ at span K	$\mathcal{O}(N)$	$\mathcal{O}(1)$
First Negative	Fixed	Queue of negatives; peek front	$\mathcal{O}(N)$	$\mathcal{O}(K)$
Window Max	Fixed	Monotonic decreasing deque of indices	$\mathcal{O}(N)$	$\mathcal{O}(K)$
Anagrams	Fixed	Frequency match comparison at span $ P $	$\mathcal{O}(N)$	$\mathcal{O}(1)$
Sum = K	Variable	Expand R ; shrink L while sum $> K$	$\mathcal{O}(N)$	$\mathcal{O}(1)$
$\leq K$ Distinct	Variable	Frequency map; shrink while size $> K$	$\mathcal{O}(N)$	$\mathcal{O}(K)$
No Repeating	Variable	Visited table; shrink while $S[R]$ duplicate	$\mathcal{O}(N)$	$\mathcal{O}(1)$
Min Window	Variable	Match count; shrink L while valid	$\mathcal{O}(N)$	$\mathcal{O}(1)$

Universal Principle: Sliding window eliminates redundant recomputations by maintaining an incremental running state. Fixed windows slide by eviction; variable windows dynamically expand and contract to guarantee linear $\mathcal{O}(N)$ execution.

Idea 6

Prefix Sums & Difference Arrays

Discrete Calculus on Arrays, $\mathcal{O}(1)$ Range Queries & Boundary Impulses

Chapter 01: Arrays | Interview Problem Solving Masterclass

The Range Query & Range Update Dilemma

Problem 1: Range Sum Queries

Given $A[0 \dots N - 1]$, answer Q queries:

Target: $\text{Sum}(L, R) = \sum_{k=L}^R A[k]$

- **Naive Scan:** $L \rightarrow R$ in $\mathcal{O}(N)$ per query.
- **Total Cost:** $\mathcal{O}(Q \cdot N) \approx 10^{10} \Rightarrow \text{TLE!}$

Problem 2: Range Modifications

Given U updates: add V to all $A[L \dots R]$:

Target: $A[k] += V \quad \forall k \in [L, R]$

- **Naive Loop:** Modify in $\mathcal{O}(N)$ per update.
- **Total Cost:** $\mathcal{O}(U \cdot N) \approx 10^{10} \Rightarrow \text{TLE!}$

The Algorithmic Mission: Can we answer Range Sum Queries in $\mathcal{O}(1)$ time, and apply Range Updates in $\mathcal{O}(1)$ time? **Yes** — via **Discrete Calculus**.

Mathematical Mental Model: Discrete Calculus

Calculus vs. Array Analogy

- **Continuous Function** $f(x) \longleftrightarrow$ **Discrete Array** $A[i]$.
- **Derivative** $\frac{df}{dx} \longleftrightarrow$ **Difference Array**
 $\Delta A[i] = A[i] - A[i - 1]$.
Measures local rate of change / boundary impulses.
- **Integral** $\int_0^x f(t)dt \longleftrightarrow$ **Prefix Sum**
 $P[i] = \sum_{k=0}^i A[k]$.
Measures total accumulated volume / area.

The Fundamental Theorem

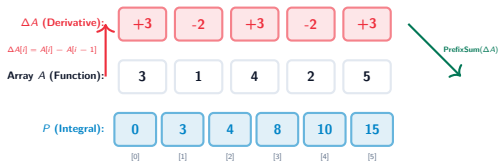
Difference and Prefix Sum are **exact mathematical inverses** of each other:

$$\text{PrefixSum}(\Delta A)[i] = A[i]$$

$$\Delta(\text{PrefixSum}(A))[i] = A[i]$$

- **Differentiation:** Concentrates range updates into 2 discrete boundary points.
- **Integration:** Restores continuous array values from boundary deltas in a single linear sweep!

Discrete Calculus Duality: Visual Architecture



Three Levels of Representation

- **Derivative (ΔA):** Sparse boundary impulses. Ideal for modifications.
- **Function (A):** The original array values.
- **Integral (P):** Cumulative monotonic staircase. Ideal for range sum queries.

Identification Framework: Decision Matrix

Problem Signal	Core Operation	Optimal Technique	Complexity
Static Array Sums	Q Queries: $\sum_{k=L}^R A[k]$	Prefix Sum Array (P)	$\mathcal{O}(N + Q)$
Batch Interval Updates	U Updates: $A[L..R] += V$	Difference Array (ΔA)	$\mathcal{O}(N + U)$
Range Parity / Bulbs	Q Range Flips: $0 \leftrightarrow 1$	Parity Difference Array	$\mathcal{O}(N + Q)$
Timeline Overlaps	Passenger / Meeting Overlaps	Event Sweep-Line (ΔA)	$\mathcal{O}(N \log N + Q)$
Target Subarray Sums	Count Subarrays with Sum $= K$	Prefix Sum + Hash Map	$\mathcal{O}(N)$

Golden Rule: If queries ask for **aggregates of static intervals**, integrate (Prefix Sum). If operations require **modifying intervals in bulk**, differentiate (Difference Array).

Foundation Tool 1: The Prefix Sum Array

1-Based Indexing Superpower

Define prefix sum array P of size $N + 1$:

$$P[0] = 0$$

$$P[i] = P[i - 1] + A[i - 1] \quad \forall i \in [1, N]$$

The $\mathcal{O}(1)$ Range Query:

$$\sum_{k=L}^R A[k] = P[R + 1] - P[L]$$

- When $L = 0$:
 $P[R + 1] - P[0] = P[R + 1] - 0$.
- Zero Edge Cases:** No 'if ($L == 0$)' branches!

Why Subtraction Works

- $P[R + 1]$ holds the sum of prefix $A[0 \dots R]$.
- $P[L]$ holds the sum of prefix $A[0 \dots L - 1]$.
- Subtracting the two cancels out the shared prefix $A[0 \dots L - 1]$, leaving **exactly** $A[L \dots R]$!

$$\underbrace{(A_0 + \dots + A_{L-1})}_{P[L]} + \underbrace{(A_L + \dots + A_R)}_{\text{Target Sum}} = \underbrace{P[R + 1]}_{\text{Total}}$$

Prefix Sum: Java Implementation

```
1 public class PrefixSum {
2     private final long[] P;
3
4     public PrefixSum(int[] a) {
5         P = new long[a.length + 1];
6         for (int i = 0; i < a.length; i++) {
7             P[i + 1] = P[i] + a[i];
8         }
9     }
10
11     public long query(int L, int R) {
12         return P[R + 1] - P[L]; // O(1)
13     }
14 }
```

Key Highlights

- **Line 4:** Array P allocated of size $N + 1$ using 'long' to prevent 32-bit integer overflow.
- **Line 8:** Precomputes all prefix sums in $\mathcal{O}(N)$ time.
- **Line 13:** Direct subtractive range query in $\mathcal{O}(1)$ time.
- **Space:** $\mathcal{O}(N)$ auxiliary array.

Prefix Sum: Visual Range Subtraction

Trace: $A = [3, 1, 4, 2, 5]$

- **Prefix Array:** $P = [0, 3, 4, 8, 10, 15]$.
- **Query:** $\text{sum}(L = 2, R = 4)$ on range $[2 \dots 4]$.
- **Target:** $A[2] + A[3] + A[4] = 4 + 2 + 5 = 11$.
- **Calculation:**
 $P[R + 1] - P[L] = P[5] - P[2] = 15 - 4 = 11$
- **Operations:** 1 subtraction ($\mathcal{O}(1)$).

$$P[R + 1] = P[5] = \sum_{k=0}^4 A[k] = 15$$

$$P[L] = P[2] = 4$$

$$\text{RangeSum}(2, 4) = P[5] - P[2] = 11$$

Takeaway: Prefix sum converts arbitrary range sum queries into constant-time endpoint evaluation.

Foundation Tool 2: The Difference Array

Definition & Construction

For array $A[0 \dots N - 1]$, define difference array $D[0 \dots N]$:

$$D[0] = A[0]$$

$$D[i] = A[i] - A[i - 1] \quad \forall i \in [1, N - 1]$$

The $\mathcal{O}(1)$ Range Update Rule:

To add value V to all elements in range $[L, R]$:

$$D[L] += V$$

$$D[R + 1] -= V \quad (\text{if } R + 1 < N)$$

Why the Dual Impulse Works

- $+V$ at index L acts as a **rising step** that propagates forward during integration.
- $-V$ at index $R + 1$ acts as a **falling step** that cancels the $+V$ propagation for all indices $> R$.
- **Reconstruction:** Sweeping a single prefix sum over D restores the updated array in $\mathcal{O}(N)$ time!

Difference Array: Java Implementation

```
1 public class DifferenceArray {
2     private final int[] D;
3     private final int N;
4     public DifferenceArray(int n) {
5         this.N = n;
6         this.D = new int[n + 1];
7     }
8     public void update(int L, int R, int val) {
9         D[L] += val;
10        if (R + 1 < N) D[R + 1] -= val;
11    }
12    public int[] reconstruct() {
13        int[] res = new int[N];
14        res[0] = D[0];
15        for (int i = 1; i < N; i++) {
16            res[i] = res[i - 1] + D[i];
17        }
18        return res;
19    }
20 }
```

Key Highlights

- **Lines 8–11:** Constant-time $\mathcal{O}(1)$ range update modifying 2 boundaries.
- **Lines 12–19:** Single linear prefix pass in $\mathcal{O}(N)$ reconstructs values.
- **Batch Efficiency:** U updates +1 reconstruction takes $\mathcal{O}(U + N)$ total time!

Difference Array: Impulse Cancellation

Trace: Update $[2 \dots 4] += 3$ on $N = 7$

- Set $D[2] += 3$ (start impulse).
- Set $D[5] -= 3$ (cancelling impulse).
- **Integration Prefix Sweep:**
 - $i = 0, 1$: $\text{sum} = 0$.
 - $i = 2$: $\text{sum} = 0 + 3 = 3$.
 - $i = 3$: $\text{sum} = 3 + 0 = 3$.
 - $i = 4$: $\text{sum} = 3 + 0 = 3$.
 - $i = 5$: $\text{sum} = 3 - 3 = 0$.
- Result: $[0, 0, 3, 3, 3, 0, 0]!$

Diff Array D :



Prefix $\sum D$:



Core Power: We turned an $\mathcal{O}(N)$ range modification loop into two $\mathcal{O}(1)$ boundary assignments.

Variation 1: Range Bulb Flipping (Parity Inversion)

Problem Statement

You have N light bulbs numbered $0 \dots N-1$, initially all **OFF** (0).

You receive Q queries: each query (L, R) toggles/flips every bulb in range $[L, R]$ ($0 \rightarrow 1$ and $1 \rightarrow 0$).

Goal: Determine the final state of each bulb and find the total number of bulbs that are **ON**.

The Parity Insight

- A bulb flipped K times is **ON** $\iff K$ is **ODD** ($K \bmod 2 == 1$).
- A bulb flipped K times is **OFF** $\iff K$ is **EVEN** ($K \bmod 2 == 0$).
- We don't need to track bulb states continuously — we only need the **cumulative flip count parity** at each bulb!

Range Bulb Flipping: Difference Mechanics

Boundary Delta Updates

For each flip query $[L, R]$:

$$\text{diff}[L] += 1$$

$$\text{diff}[R + 1] -= 1$$

- Process all Q queries in $\mathcal{O}(Q)$ time.
- No array elements in $[L + 1 \dots R]$ are touched during query execution!

Single Pass Prefix Sweep

Maintain running sum across the array:

$$\text{runningFlips} = \sum_{k=0}^i \text{diff}[k]$$

$$\text{Bulb}[i] = (\text{runningFlips} \pmod 2 == 1) ? 1 : 0$$

- Resolves all N bulbs in a single $\mathcal{O}(N)$ linear sweep.
- Total Time: $\mathcal{O}(N + Q)$ instead of $\mathcal{O}(N \cdot Q)$!

Range Bulb Flipping: Java Implementation

```
1 public static int countBulbsOn(  
2     int n, int[][] queries) {  
3     int[] diff = new int[n + 1];  
4     for (int[] q : queries) {  
5         diff[q[0]]++;  
6         if (q[1] + 1 < n) diff[q[1] + 1]--;  
7     }  
8     int runningFlips = 0, onCount = 0;  
9     for (int i = 0; i < n; i++) {  
10        runningFlips += diff[i];  
11        if ((runningFlips & 1) == 1) {  
12            onCount++;  
13        }  
14    }  
15    return onCount;  
16 }
```

Key Highlights

- **Lines 5–6:** Ingests each range flip query in $\mathcal{O}(1)$ time.
- **Line 10:** Integrates flip impulses into cumulative flip count.
- **Line 11:** Fast bitwise parity check $(\text{runningFlips} \& 1) == 1$.
- **Complexity:** $\mathcal{O}(N + Q)$ time, $\mathcal{O}(N)$ space.

Range Bulb Flipping: Visual State Trace

Trace: $N = 6$, Queries $[1, 4]$ and $[3, 5]$

- **Query 1** $[1, 4]$: $\text{diff}[1]++$, $\text{diff}[5]--$.
- **Query 2** $[3, 5]$: $\text{diff}[3]++$.
- **Sweep Prefix Sums:**
 - Bulb 0: 0 flips $\implies$ **OFF**.
 - Bulb 1: 1 flip $\implies$ **ON**.
 - Bulb 2: 1 flip $\implies$ **ON**.
 - Bulb 3: 2 flips $\implies$ **OFF**.
 - Bulb 4: 2 flips $\implies$ **OFF**.
 - Bulb 5: 1 flip $\implies$ **ON**.
- **Final Bulbs ON:** 3 (indices 1, 2, 5).

Total Flips:	0	1	1	2	2	1
	[0]	[1]	[2]	[3]	[4]	[5]
Bulb State:	OFF	ON	ON	OFF	OFF	ON

Correctness Invariant: Bulb i reflects the combined parity of all overlapping intervals covering index i .

Variation 2: Timeline Events — Car Pooling

Problem Statement

A vehicle with capacity C travels along a single 1D route.

Given $\text{trips}[i] = [\text{numPassengers}, \text{from}, \text{to}]$, return **true** if and only if it is possible to complete all trips without exceeding capacity C at any point.

Constraint: Passengers drop off at to *before* new pickups at that mile.

Discrete Event Sweep-Line

- Pickup at $\text{from} \implies \Delta[\text{from}] += \text{passengers}$.
- Dropoff at $\text{to} \implies \Delta[\text{to}] -= \text{passengers}$.
- The prefix sum $\sum_{k=0}^x \Delta[k]$ represents the **instantaneous passenger load** at coordinate x .
- Invariant: Check if $\max(\text{load}[x]) \leq C$.

Car Pooling: Java Implementation

```
1 public static boolean carPooling(  
2     int[][] trips, int capacity) {  
3     int[] timeline = new int[1001];  
4     for (int[] t : trips) {  
5         timeline[t[1]] += t[0]; // pickup  
6         timeline[t[2]] -= t[0]; // dropoff  
7     }  
8     int currentLoad = 0;  
9     for (int passengers : timeline) {  
10        currentLoad += passengers;  
11        if (currentLoad > capacity) {  
12            return false;  
13        }  
14    }  
15    return true;  
16 }
```

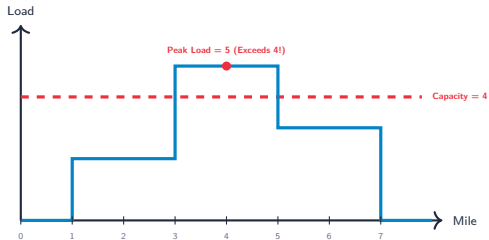
Key Highlights

- Lines 5–6: Delta accumulation at pickup and dropoff points in $\mathcal{O}(1)$.
- Line 10: Running prefix sum tracks live passenger count along road.
- Line 11: Instant capacity violation check.
- Complexity: $\mathcal{O}(N + \text{MaxMile})$ time, $\mathcal{O}(\text{MaxMile})$ space.

Car Pooling: Visual Timeline Trace

Trace: Capacity = 4

- Trip 1:
[2 pax, 1 → 5] $\Rightarrow \Delta[1] += 2, \Delta[5] -= 2$.
- Trip 2:
[3 pax, 3 → 7] $\Rightarrow \Delta[3] += 3, \Delta[7] -= 3$.
- Prefix Sum State:
 - Mile 1: Load = $2 \leq 4$ (OK).
 - Mile 3: Load = $2 + 3 = 5 > 4$ (OVERFLOW!).
- Result: Return false.



Generalization: This sweep-line pattern solves Meeting Rooms II, Corporate Flight Bookings, and CPU task scheduling.

Variation 3: Subarray Sum Equals K

Problem Statement

Given an integer array `nums` (which may contain **negative numbers**) and integer K , find the total number of contiguous subarrays whose sum equals K .

The Trap: Why Two Pointers Fails!

- Negative numbers break monotonicity.
- Expanding right pointer can decrease sum; shrinking left pointer can increase sum.

Algebra in Prefix Space

A subarray from i to j has sum:

$$\text{sum}(i \dots j) = P[j] - P[i - 1] = K$$

Rearranging algebraically:

$$P[i - 1] = P[j] - K$$

- As we compute $P[j]$, we query: *"How many prior prefix sums were equal to $P[j] - K$?"*
- Solved via **Prefix Sum + Hash Map!**

Subarray Sum Equals K: Java Implementation

```
1 public static int subarraySum(int[] nums, int k) {
2     Map<Integer, Integer> map = new HashMap<>();
3     map.put(0, 1); // Base case: empty prefix
4     int prefixSum = 0, count = 0;
5     for (int x : nums) {
6         prefixSum += x;
7         int target = prefixSum - k;
8         count += map.getOrDefault(target, 0);
9         map.put(prefixSum,
10                map.getOrDefault(prefixSum, 0) + 1);
11     }
12     return count;
13 }
```

Key Highlights

- **Line 3:** 'map.put(0, 1)' handles subarrays starting from index 0 ($P[j] == K$).
- **Line 7:** Computes required complement $P[j] - K$.
- **Line 8:** Adds frequency of matching historical prefixes in $\mathcal{O}(1)$.
- **Complexity:** $\mathcal{O}(N)$ time, $\mathcal{O}(N)$ space.

Subarray Sum Equals K: State Evolution

Step	Num	PrefixSum	Target ($P - K$)	Map Lookup	Count	Map State
Init	—	0	—	—	0	$\{0 : 1\}$
$i = 0$	1	1	$1 - 2 = -1$	0	0	$\{0 : 1, 1 : 1\}$
$i = 1$	2	3	$3 - 2 = 1$	1	1	$\{0 : 1, 1 : 1, 3 : 1\}$
$i = 2$	3	6	$6 - 2 = 4$	0	1	$\{0 : 1, 1 : 1, 3 : 1, 6 : 1\}$
$i = 3$	-2	4	$4 - 2 = 2$	0	1	$\{0 : 1, 1 : 1, 3 : 1, 6 : 1, 4 : 1\}$
$i = 4$	1	5	$5 - 2 = 3$	1	2	$\{0 : 1, 1 : 1, 3 : 1, 6 : 1, 4 : 1, 5 : 1\}$

Found Subarrays ($K = 2$): $[1, 2]$ at indices $(0 \dots 1)$ and $[3, -2, 1]$ at indices $(2 \dots 4)$. Both resolved in a single linear pass!

Variation 4: Corporate Flight Bookings

Problem Statement

There are N flights labeled $1 \dots N$.

Given bookings records
 $\text{bookings}[i] = [\text{first}, \text{last}, \text{seats}]$, each booking
reserves seats on every flight from first to last
inclusive.

Goal: Return an array of length N representing the
total reserved seats on each flight.

Difference Mapping

- Map 1-based flights to 0-based array:

$$\Delta[\text{first} - 1] += \text{seats}$$

$$\Delta[\text{last}] -= \text{seats} \quad (\text{if } \text{last} < N)$$

- In-place prefix sum accumulation directly in the result array!
- Space: $\mathcal{O}(1)$ auxiliary memory.

Flight Bookings: Java Implementation

```
1 public static int[] corpFlightBookings(  
2     int[][] bookings, int n) {  
3     int[] ans = new int[n];  
4     for (int[] b : bookings) {  
5         ans[b[0] - 1] += b[2];  
6         if (b[1] < n) ans[b[1]] -= b[2];  
7     }  
8     for (int i = 1; i < n; i++) {  
9         ans[i] += ans[i - 1]; // In-place prefix  
10    }  
11    return ans;  
12 }
```

Key Highlights

- **Lines 4–6:** Applies all booking reservations in $\mathcal{O}(Q)$ time.
- **Lines 8–10:** In-place prefix accumulation modifies the return array directly.
- **Total Time:** $\mathcal{O}(N + Q)$ optimal.
- **Auxiliary Space:** $\mathcal{O}(1)$ extra memory.

Complexity Analysis: Discrete Calculus

Technique	Build / Preprocess	Point / Range Query	Range Update	Space
Naive Array	$\mathcal{O}(1)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(1)$
Prefix Sum Array	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Difference Array	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Prefix + Hash Map	$\mathcal{O}(N)$ (online)	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$

Competitive Trade-Off: Prefix sum trades preprocessing time for instant query time. Difference array trades reconstruction time for instant batch range modifications.

Master Discrete Calculus Pattern Matrix

Archetype	Operator	Boundary / State Invariant	Time	Space
Range Sum	Prefix (P)	$\text{Sum}(L, R) = P[R + 1] - P[L]$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Range Add	Diff (ΔA)	$+V$ at L , $-V$ at $R + 1$	$\mathcal{O}(1)$	$\mathcal{O}(N)$
Bulb Flips	Parity Diff	$+1$ at L , -1 at $R + 1 \implies$ odd flips	$\mathcal{O}(N+Q)$	$\mathcal{O}(N)$
Timeline	Event Sweep	$\text{In} + C$, $\text{Out} - C \implies \text{Load} \leq \text{Cap}$	$\mathcal{O}(N+Q)$	$\mathcal{O}(N)$
Subarray $= K$	Prefix + Map	Query $P[j] - K$ frequency in $\mathcal{O}(1)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$

Universal Principle: Continuous calculus translates to discrete arrays via difference impulses and cumulative prefix integrals. Mastering this duality converts quadratic range bottlenecks into linear-time pipelines.

Session Summary

1. **The Partition Engine:** Lomuto partitioning uses dual pointers (i boundary, j scanner) to segregate elements by arbitrary predicates in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space.
2. **Custom Comparators:** The 3-state comparator contract ($\text{sgn} < 0, = 0, > 0$) establishes strict weak orderings. Multi-field tiebreakers and suffix reversals enforce global invariants.
3. **Binary Search & Bounds:** Logarithmic search halves ordered search spaces in $\mathcal{O}(\log N)$. Lower and upper bounds capture frequency intervals and discrete duplicate ranges.
4. **Two-Pointer Techniques:** Converging and fast-slow pointers eliminate quadratic candidate spaces in $\mathcal{O}(N)$ time by exploiting sorted monotonicity and bottleneck bounds.
5. **Sliding Window Invariants:** Replaces $\mathcal{O}(N \cdot K)$ recomputation with $\mathcal{O}(1)$ edge updates ($+A[R], -A[L]$). Monotonic dequeues maintain sliding window extrema in $\mathcal{O}(N)$ amortized time.
6. **Discrete Calculus on Arrays:** Prefix sum acts as discrete integration ($\mathcal{O}(1)$ static range sums); difference array acts as discrete differentiation ($\mathcal{O}(1)$ interval updates).

Data Structures & Algorithms Masterclass — Chapter Learning Journey